

Gỡ lỗi mã ML

ONE LOVE. ONE FUTURE.

1

Gỡ lỗi là một quá trình khó khăn nhưng cần thiết



2

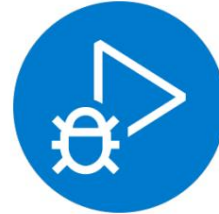
Gỡ lỗi mã ML thậm chí còn khó hơn

- Lỗi trong mã ML có thể không dành riêng cho ML và có thể dành riêng cho

ML • Lỗi cổ điển: Mã không chạy

- Sử dụng trình gỡ lỗi truyền thống để tìm những

- Lỗi cụ thể của ML: Mô hình của tôi không hội tụ • Cần có phương pháp tiếp cận chính xác để gỡ lỗi



Hãy bắt đầu với những tác phẩm kinh điển.



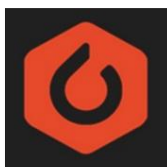
Bước đầu tiên: Có một môi trường tốt

💡 Sổ tay Jupyter rất tuyệt vời trong mục đích của chúng: khám phá ý tưởng và kết hợp mã + văn bản thành tài liệu độc lập.

💡 Tuy nhiên, việc gỡ lỗi mã trong sổ tay có thể rất khó khăn.



Mã VS



Torchstudio



Cây Pycharm



Người do thần

Gỡ lỗi Python nói chung

1. In báo cáo

```
print("x.shape = {}".format(x.shape))
```

2. Dừng lại ở một điểm thú vị trong mã hóa và tương tác

```
from IPython import embed; embed()
... # do your stuff interactively here
exit() # exit ipython to let your code continue
```

3. Làm việc trong trình gỡ lỗi thực tế

```
import pdb; pdb.set_trace()
```

Tìm hiểu thêm tại đây

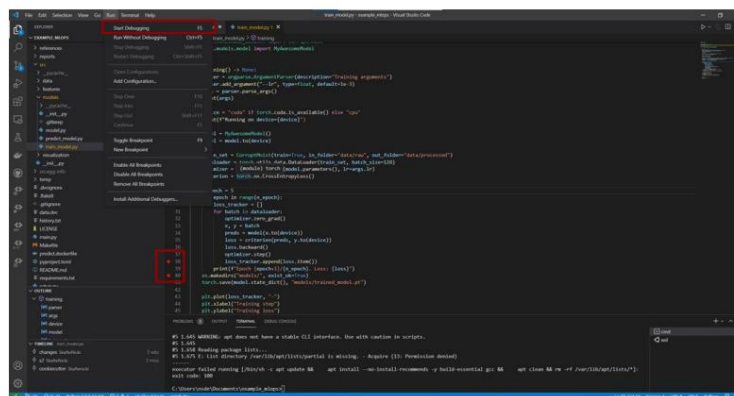
<https://switowski.com/blog/ipython-gỡ-lỗi>



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

5

Trình gỡ lỗi mã VS



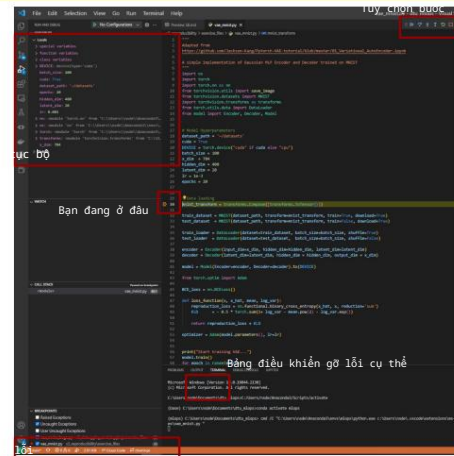
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

6

Trình gỡ lỗi mã VS

Tùy chọn bước:

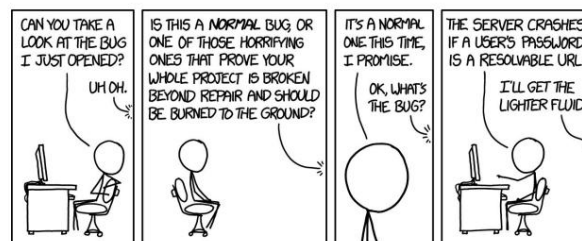
- F5: điểm dừng tiếp theo
- F10: dòng tiếp theo
- F11: bước vào
- Shift-F11: bước ra ngoài
- CTRL-Shift-F11: Khởi động lại
- Shift-F5: dừng lại



Quay lại những lỗi cụ thể của ML

Hoặc tốt hơn được biết đến như

⚠ Khi mọi thứ đều chạy nhưng kết quả lại sai



Việc tìm ra những con bọ này đòi hỏi kinh nghiệm. Một hỗn hợp

những phát hiện của tôi trong vài năm trở lại đây và nhiều năm khác.

Các bước gỡ lỗi ML

1. Kiểm tra dữ liệu của bạn
2. Bắt đầu đơn giản nhất có thể
3. Làm cho mọi thứ mang tính xác định
4. Nghiên cứu toán học
5. Phù hợp với dữ liệu của bạn
6. Xem xét tổn thất của bạn
7. Hình dung mọi thứ
8. Thêm độ phức tạp theo từng bước



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

9

1. Kiểm tra dữ liệu của bạn!

Điểm khởi đầu của bạn luôn phải là dữ liệu trong ML!

Kiểm tra

- ✓ Kiểm tra số liệu thống kê tóm tắt (dữ liệu đã được chuẩn)
- ✓ (??) Xem xét phân phối nhãn (có bị xáo trộn không?)
- ✓ Hình dung một vài mẫu (chúng có bị hỏng theo bất kỳ cách nào không?)

Nếu bạn đang làm việc trên các tập dữ liệu bạn biết, bạn có thể bỏ qua các bước này.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

10

2. Bắt đầu đơn giản nhất có thể

Loại bỏ tất cả những thứ
rườm rà • Độ chính
xác hỗn hợp • Chuẩn hóa như bỏ
học • Dừng sớm •
Học trình lập lịch tốc độ
• .

Một hoặc nhiều trong số những điều này có thể được bật theo
mặc định nếu bạn bắt đầu từ cơ sở mã của người khác



Lời nhắc:

Bot học máy với chiếc mũ lạ mắt

3. Làm cho mọi thứ mang tính quyết định

Mỗi lần chạy máy học theo mặc định là ngẫu nhiên. Hãy thử sửa:

- 💡 Gieo hạt mọi thứ và sử dụng cùng một hạt giống ở mọi nơi
- 💡 Xóa tất cả dữ liệu tăng cường
- 💡 Chỉ sử dụng một lô dữ liệu duy nhất mà bạn có cảm nhận về kết quả

```
int getRandomNumber()  
{  
    return 4; // chosen by fair dice roll.  
             // guaranteed to be random.  
}
```

4. Nghiên cứu toán học

Gỡ lỗi toán học:

- 💡 Xem lại mã của bạn, từng dòng một
- 💡 Phải có sự khớp một-một giữa các phương trình và các dòng mã
- 💡 Hãy chỉnh sửa lại nếu không rõ ràng

Kiểm tra kích thước và chú thích nếu cần thiết hoặc sử dụng [phần mềm đánh máy](#)

```
# add shape comments
A = torch.randn(N, D) # Nx D
x = torch.randn(D) # D
Ax = A.mv(x) # N
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

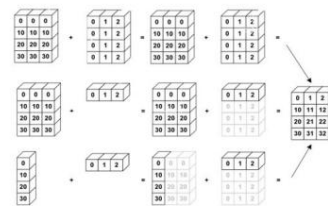
13

4. Nghiên cứu toán học (tiếp theo)

💡 Hãy chú ý theo dõi chương trình phát sóng!

💡 Phát sóng trong python vừa là một điều may mắn vừa là một điều bất hạnh

💡 Nó có thể tạo ra vấn đề (ví dụ thực tế)



```
import torch
preds = torch.randn(100,)
target = torch.randn(100,1)
loss = (preds - target).abs().pow(2.0).sum()
```

Vấn đề ở đây là gì?



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

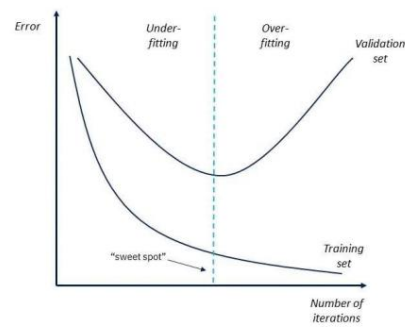
14

5. Đánh quá mức có tốt không?

Quá mức thường được coi là lỗi của túi.

💡 Các mô hình phải có khả năng ghi nhớ một lô hàng

💡 Đào tạo trên một lô, nếu tổn thất là 0 di chuyển tiếp đến các mô hình lớn hơn/dữ liệu lớn nếu không thì gỡ lỗi



6. Thực sự nhìn vào mắt mắt của bạn

• Giả sử mô hình của bạn đang đào tạo mà không có lỗi, nhưng mắt mắt của bạn không hoạt động như mong đợi. • Bạn có in/ghi lại kết quả chính

xác không? • Bạn có nhớ `loss.backward`, `optimizer.step` và `optimizer.zero_grad` không? • Tốc độ học và kích thước lô của bạn thì sao?

• Đối với tổn thất của bạn, nếu có thể, hãy tính toán trong không gian logarit

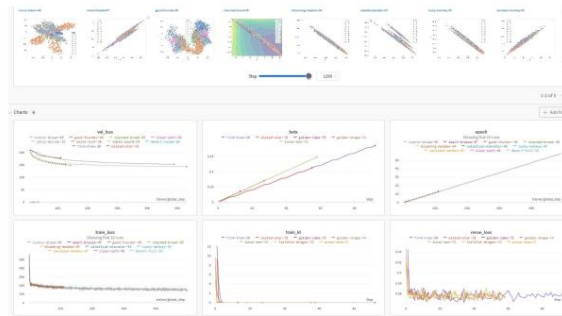
```
a = 1
for x in data:
    a = a * x

log_a = 0
for x in data:
    log_a = log_a + log(x)
```


7. VisualizaSons là bạn của bạn

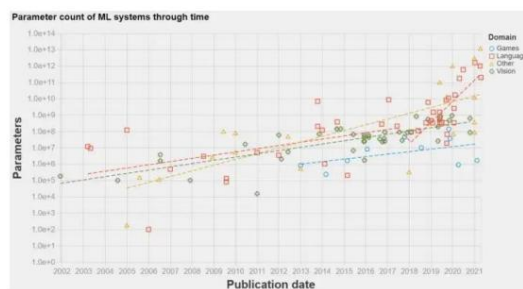
Ghi nhật ký và hình dung mọi thứ

- 💡 Sự mất mát do luyện tập thực sự đang giảm đi phải không?
- 💡 Bạn có thể thêm số liệu bổ sung không?
- 💡 Nhật ký thay đổi siêu tham số động (tốc độ học với trình lập lịch lr)
- 💡 Biểu đồ dữ liệu, dự đoán, tái thiết, v.v. theo thời gian



8. Tăng độ phức tạp theo thời gian

- Nếu bạn vẫn ổn, thì • Có một đường cơ sở chỉ hoạt động (=đào tạo trên nhiều dữ liệu hơn) • Dừng việc lắp quá mức:
- Thêm lại quy tắc hóa • Thêm lại phần tăng cường dữ liệu
 - Điều chỉnh các tham số siêu



Gỡ lỗi

- Gỡ lỗi là một kỹ năng được cải thiện theo kinh nghiệm
- Phương pháp gỡ lỗi phổ biến nhưng không hiệu quả: câu lệnh `print(...)`
- Gỡ lỗi hiệu quả cần thiết cho cơ sở mã lớn
- Giải pháp: Làm quen với trình gỡ lỗi tích hợp của Python, `pdb`

Gọi Python Debugger

- Đặt theo dõi với `pdb` bằng cách chèn:

```
import pdb
pdb.set_trace()
```
- Sử dụng lệnh để thực hiện từng bước mã (xem hướng dẫn)

Gỡ lỗi trong trình soạn thảo

- Trong VS Code, sử dụng điểm dừng nội tuyến (F9) để dừng thực thi
- Thực thi tập lệnh ở chế độ gỡ lỗi
- Điểm dừng nội tuyến = các chấm đỏ nhỏ ở bên trái mã



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

21

Gỡ lỗi với lỗi

- Tự động khởi động trình gỡ lỗi khi xảy ra lỗi: `python -m pdb -c continue my_script.py`



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

22

Tổng quan về Profilers

- Hồ sơ tập trung vào việc cải thiện hiệu suất mã • Trong phiên này: tập trung vào thời gian chạy
 - Hai câu hỏi chính: •
 - Mỗi phương thức được gọi bao nhiêu lần? •
 - Mỗi phương thức mất bao lâu? • Profiler
- giúp xác định các nút thắt trong mã



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

25

Tổng quan về cProfile

- Trình tạo hồ sơ Python tích hợp • Cung cấp tổng quan về thời gian chạy hàm và phương thức
- Cách sử dụng:

```
python -m cProfile -o <tệp_đầu_ra> -s <thứ_tự_sắp_xếp> myscript.py
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

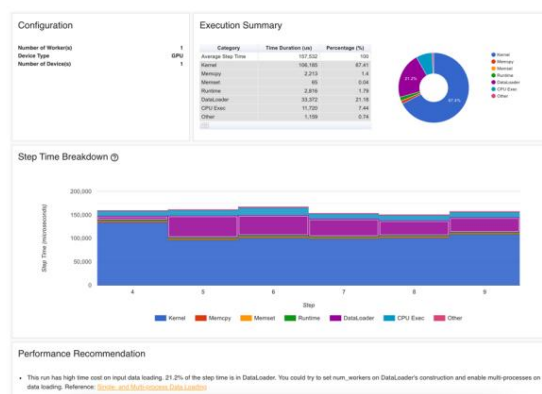
26

Bài tập cho cProfile

1. Phân tích kết quả đầu ra: Hàm nào mất nhiều thời gian chạy nhất?
 2. Sự khác biệt giữa tottime và cumtime
 3. Trực quan hóa hồ sơ với snakeviz 4.
- Tối ưu hóa quá trình chạy và xác minh tốc độ

Hồ sơ PyTorch

- Việc lập hồ sơ mã ML liên quan đến các thiết bị hỗn hợp (CPU+GPU) • Mục đích: Xác định các điểm nghẽn trong các phép tính chồng chéo • PyTorch có trình lập hồ sơ riêng cho các ứng dụng phức tạp



Ví dụ về PyTorch Profiler

- Ví dụ: với `torch.profiler.profile(...)` làm prof: đầu ra = model(data) • Theo dõi thời gian hạt nhân và thời gian truyền (ví dụ: memcpy) • Cung cấp các khuyến nghị

Bài tập cho PyTorch Profiler

1. Khám phá [API cho trình tạo hồ sơ](#)
2. Chạy ví dụ tạo hồ sơ bằng ResNet18
3. Tìm kiếm các phép toán chiếm nhiều CPU nhất
4. (Optional) Tạo hồ sơ bằng GPU

<https://pytorch.org/docs/stable/profiler.html>

Ví dụ

```
nhập torch nhập
torchvision.models làm mô hình từ torch.profiler
nhập hồ sơ, ProfilerActivity
model = models.resnet18() đầu vào =
torch.randn(5, 3, 224, 224) với
profile(activities=[ProfilerActivity.CPU], record_shapes=True) làm prof:
model(inputs)
```

Tối ưu hóa với PyTorch Profiler

- Hồ sơ sử dụng bộ nhớ với các đối số bổ sung
- Xuất kết quả lập hồ sơ với:

```
prof.export_chrome_trace("dấu vết.json")
```
- Trực quan hóa bằng cách theo dõi Chrome (chrome://tracing)

Hồ sơ PyTorch nâng cao

- Hồ sơ nhiều lần lặp lại để tránh sai lệch mẫu đơn: với `profile(...)` là
prof: đối với i trong phạm vi(10):
 model(inputs) prof.step()



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

33

Hiển thị kết quả với TensorBoard

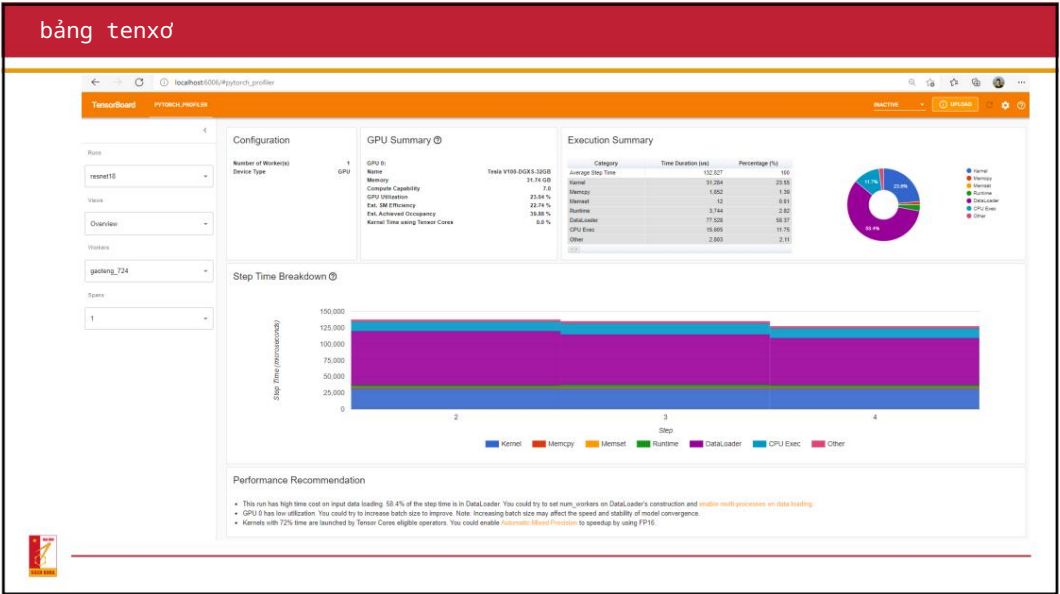
```
từ torch.profiler nhập hồ sơ,  
tensorboard_trace_handler với hồ sơ (...,  
on_trace_ready=tensorboard_trace_handler("./log/resnet18")) làm chuyên gia:  
...
```

- Sử dụng TensorBoard để trực quan hóa hồ sơ:
 tensorboard --logdir=./log
- Khám phá giao diện và so sánh các lần chạy bằng cách sử dụng tab diff



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

34



35



36

Ghi nhật ký là gì?

- Mô-đun cốt lõi trong phát triển ứng dụng. • Ghi lại các sự kiện và hoạt động theo thời gian.
- Lợi ích của việc ghi nhật ký đúng cách:
 - Gỡ lỗi dễ dàng hơn. • Theo dõi tình trạng và hiệu suất của ứng dụng. • Kiểm tra các hoạt động để tham khảo trong tương lai. • Phân tích hành vi của ứng dụng để có thông tin chi tiết.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

37

Các loại ghi nhật ký

- Ghi nhật ký ứng dụng: Hữu ích cho mọi loại ứng dụng. • Ghi nhật ký thử nghiệm: Cần thiết cho các dự án học máy liên quan đến thử nghiệm.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

38

Ghi nhật ký ứng dụng

- Ví dụ cơ bản: Câu lệnh in Python. **đối với** `batch_idx`,
`batch` **trong** `enumerate(data_loader): print(f"Đang xử lý batch {batch_idx} ngoài`
`{len(bộ nạp dữ liệu)}")`
- Hữu ích nhưng hạn chế; mô-đun ghi nhật ký của Python cung cấp chức năng ghi nhật ký linh hoạt hơn.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

39

Thành phần ghi nhật ký Python

1. Logger: Điểm vào của hệ thống ghi nhật ký.
2. Trình xử lý: Chuyển hướng các thông điệp nhật ký (ví dụ: bảng điều khiển, tệp).
3. Trình định dạng: Chỉ định bố cục nhật ký (ví dụ: dấu thời gian).
4. Mức độ: Thiết lập mức độ nghiêm trọng của nhật ký.



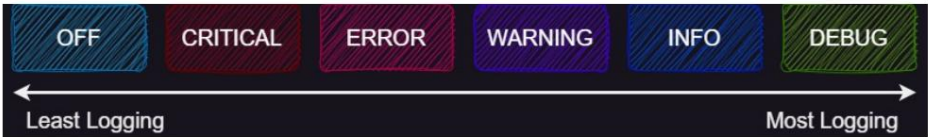
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

40

Mức độ ghi nhật ký

- Sử dụng các mức để lọc đầu ra nhật ký:


```
logger.debug("Được sử dụng để gỡ lỗi.")
logger.info("Thông báo mang tính thông tin.")
logger.warning("Có cảnh báo.") logger.error(" Đã xảy ra lỗi.") logger.critical(" Lỗi nghiêm trọng.")
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

41

Ghi nhật ký so với Xử lý lỗi

- Ghi nhật ký cung cấp thông tin sau khi chương trình chạy.
- Xử lý lỗi (thử, nâng) điều chỉnh luồng chương trình khi chạy.

```
def f(x: int): if
    not isinstance(x, int): raise
        ValueError("Expected an integer")
    trả về 2 * x
```

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

42

Ghi chép thử nghiệm

- Cần thiết để theo dõi hiệu suất của mô hình trong quá trình đào tạo. •

Các công cụ chính:

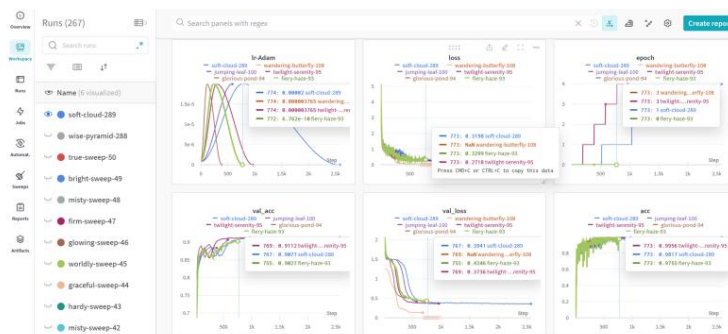
- [Tensorboard](#) •

[Trọng số & Độ lệch](#) •

[MLFlow](#)

- Trọng số và Độ lệch (wandb) được sử dụng trong khóa học này để ghi nhật ký các thí nghiệm.

Bảng điều khiển Trọng số và Độ lệch (wandb)



Bài tập: Thiết lập ghi nhật ký

1. Tạo một trình ghi nhật

ký cơ bản: `import`

```
logging.basicConfig(level=logging.DEBUG) logger =  
logging.getLogger(__name__)
```

2. Sử dụng cấu hình ghi nhật ký để xuất ra thiết bị đầu cuối và tệp.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

45

Thử nghiệm với Trọng số & Độ lệch

1. Tạo tài khoản wandb.

2. Cài đặt wandb: `pip install wandb` 3. Bây

giờ hãy kết nối với tài khoản wandb của bạn: `wandb login` 4.

Ghi nhật ký các thử nghiệm bằng cách tích hợp wandb vào mã của bạn:

```
wandb.init(dự án="ví dụ") wandb.log({"mất  
mất": giá trị mất mát})
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

46

Kiểm thử đơn vị



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

1

Mô-đun cốt lõi

- Tích hợp liên tục (CI) đảm bảo các bản cập nhật cơ sở mã được kiểm tra tự động.
- CI giúp phát hiện lỗi sớm nhưng không đảm bảo không có lỗi cơ sở mã.
- Các thử nghiệm được thực hiện trong CI rất quan trọng đối với tính hiệu quả của nó.

Martin Fowler:

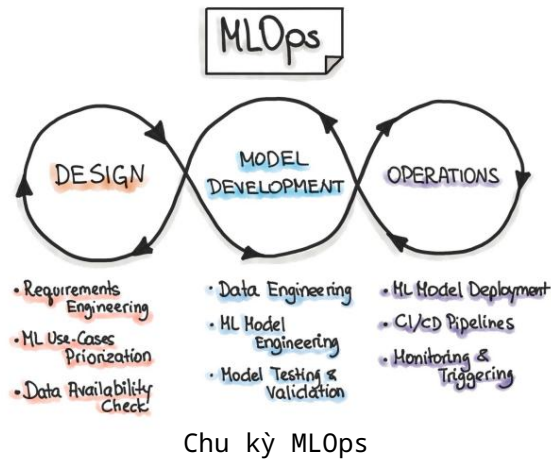
Tích hợp liên tục không loại bỏ được lỗi, nhưng nó giúp việc tìm và loại bỏ lỗi dễ dàng hơn đáng kể.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

2

- Tích hợp liên tục là nền tảng của chu trình MLOps.
- CI cấu trúc và tự động thử nghiệm nhưng chỉ mạnh bằng tự kiểm tra.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

3

Kiểm thử đơn vị

- Kiểm thử đơn vị tập trung vào việc kiểm thử từng phần riêng lẻ của mã (chức năng, mô-đun, v.v.).
- Cho phép dễ dàng cô lập phần mã bị hỏng sau khi cập nhật.
- Khác với thử nghiệm tích hợp, cũng quan trọng như vậy nhưng không phải là trọng tâm ở đây.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

4

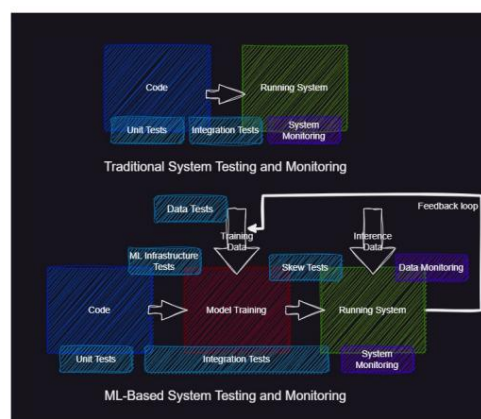
Kiểm thử đơn vị trong MLOps

- Các bài kiểm tra đơn vị rất quan trọng đối với các hệ thống học máy.
- Các hệ thống học máy phụ thuộc vào dữ liệu, làm tăng thêm tính phức tạp.
- Kiểm tra phải bao gồm:
 - Kiểm tra đơn vị
 - Kiểm tra tích hợp
 - Kiểm tra dữ liệu
 - Kiểm tra cơ sở hạ tầng
 - Giám sát phân phối dữ liệu



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

5



Độ phức tạp của hệ thống



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

6

Tổng quan về Pytest

- Pytest là một khuôn khổ được sử dụng để viết và chạy thử nghiệm trong Python.
- Cho phép thực hiện thử nghiệm dễ dàng và cung cấp kết quả thử nghiệm chi tiết.

Bài tập

1. Tạo một thư mục kiểm tra.
2. Đọc Hướng dẫn [bắt đầu sử dụng Pytest](#).
3. Cài đặt Pytest:

```
bash pip install pytest
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

7

Bài kiểm tra viết

- Tạo các tệp thử nghiệm có tên test_*.py.
- Gói

từng thử nghiệm trong một hàm bắt đầu bằng test_*:

```
# điều này sẽ được tìm thấy và thực thi bởi pytest def
```

```
test_something():
```

```
...
```

```
# điều này sẽ không được tìm thấy và thực thi bởi pytest def
```

```
something_to_test():
```

```
...
```

4. Kiểm tra xem các bài kiểm tra có vượt qua không:

```
bash
```

```
xét nghiệm pytest/
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

8

Kiểm tra dữ liệu

- Tạo tests/test_data.py. • Ví dụ về cấu trúc

kiểm tra: `def test_data():`

```
dataset = MNIST(...)
```

```
assert len(dataset) == N_train
```

để đào tạo và N_test để kiểm tra khẳng định rằng mỗi điểm dữ liệu có hình dạng [1,28,28]

hoặc [784] tùy thuộc vào cách bạn chọn định dạng khẳng định rằng tất cả các nhãn đều được biểu diễn

- Đảm bảo các điểm dữ liệu và nhãn là chính xác.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

9

Kiểm tra dữ liệu - Giải pháp

- Giải pháp

```
từ my_project.data import corrupt_mnist
```

```
def test_data(): train,
```

```
test = corrupt_mnist() assert len(train) ==
```

```
30000 assert len(test) == 5000 đối với
```

```
tập dữ liệu trong [train, test]: đối
```

```
với x, y trong tập dữ liệu: assert x.shape
```

```
== (1, 28, 28) assert y in
```

```
range(10) train_targets =
```

```
torch.unique(train.tensors[1])
```

```
assert (train_targets == torch.arange(0,10)).all() test_targets =
```

```
torch.unique(test.tensors[1]) assert (test_targets == torch.arange(0,10)).all()
```

- Đảm bảo các điểm dữ liệu và nhãn là chính xác.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

10

Kiểm tra mô hình

- Tạo tests/test_model.py.

- Cấu trúc bài kiểm tra mẫu:

```

định nghĩa test_model():
    mô hình = MyAwesomeModel()
    x = ngọn đuốc.randn(1, 1, 28, 28)
    y = mô hình(x)
    khẳng định y.shape == (1, 10) # Kiểm tra hình dạng đầu ra của mô hình

```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

11

Kiểm tra mô hình

- Ví dụ khác

```

# src/models/model.py
def forward(self, x: Tensor):
    nếu x.ndim != 4: raise
        ValueError('Dự kiến đầu vào cho tenxơ 4D')
    nếu x.shape[1] != 1 hoặc x.shape[2] != 28 hoặc x.shape[3] != 28:
        raise ValueError('Dự kiến mỗi mẫu có hình dạng [1, 28,
28]')

```

- Tốt hơn một

```

# kiểm tra/test_model.py
nhập pytest từ
my_project.model nhập MyAwesomeModel

định nghĩa test_error_on_wrong_shape():
    model = MyAwesomeModel() với
    pytest.raises(ValueError, match='Đầu vào dự kiến cho tenxơ 4D')
        model(torch.randn(1,2,3)) với
    pytest.raises(ValueError, match='Dự kiến mỗi mẫu có hình dạng [1, 28, 28]')
        model(torch.randn(1,1,28,29))

```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

12

Kiểm tra đào tạo

- Tạo `tests/test_training.py`. • Ví dụ: khẳng định hành vi đào tạo phù hợp, đưa ra cảnh báo/lỗi khi cần thiết.

```
def test_training():  
    ...
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

13

Phạm vi phủ sóng và tham số hóa

- Sử dụng `pytest.mark.parametrize` để kiểm tra các đối số đầu vào khác nhau:

```
@pytest.mark.parametrize("batch_size", [32, 64])  
def test_model(batch_size:  
int):  
    model = MyModel()  
    x = torch.randn(batch_size,  
1, 28, 28)  
    y = model(x)  
    assert y.shape == (batch_size, 10)
```

- Sử dụng phạm vi phủ sóng để đo lường mức độ cơ sở mã của bạn đang được thử nghiệm: `pip install coverage coverage run -m pytest tests/ coverage report`



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

14

Kiểm tra kiến thức

1. Việc bao phủ 100% mã có đảm bảo không có lỗi không?
 - Không, nó chỉ đảm bảo tất cả các dòng được thực thi, nhưng không đảm bảo không có lỗi.
2. Có bao nhiêu bài kiểm tra được chạy dựa trên cài đặt tham số?



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

15

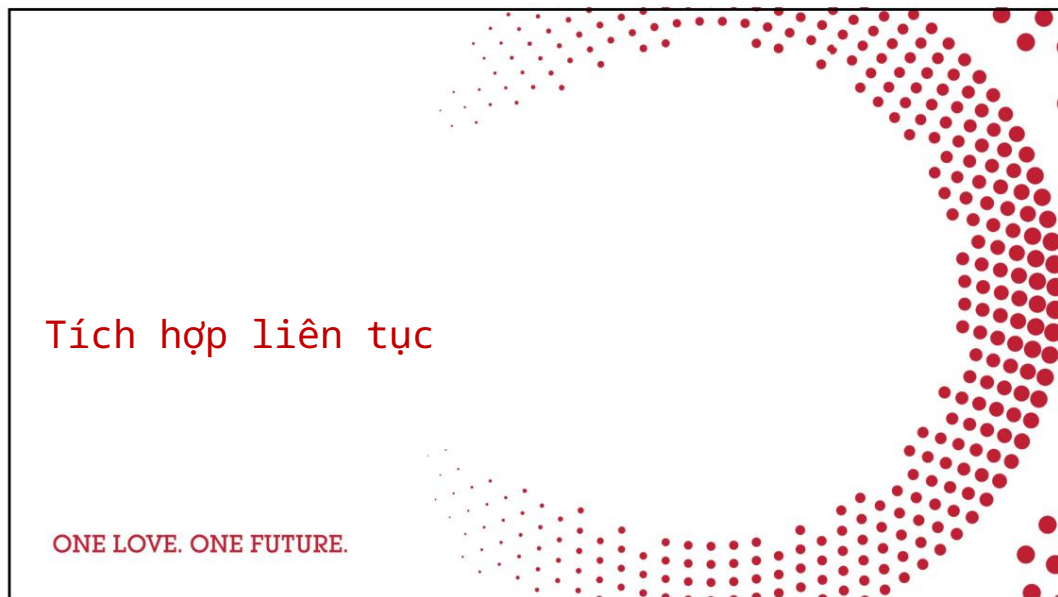
Công cụ bổ sung

- unittest: Khung kiểm thử tích hợp của Python.
- giả thuyết: Giúp tìm lỗi trong một số trường hợp.
- doctest: Kiểm thử các ví dụ mã có trong tài liệu.

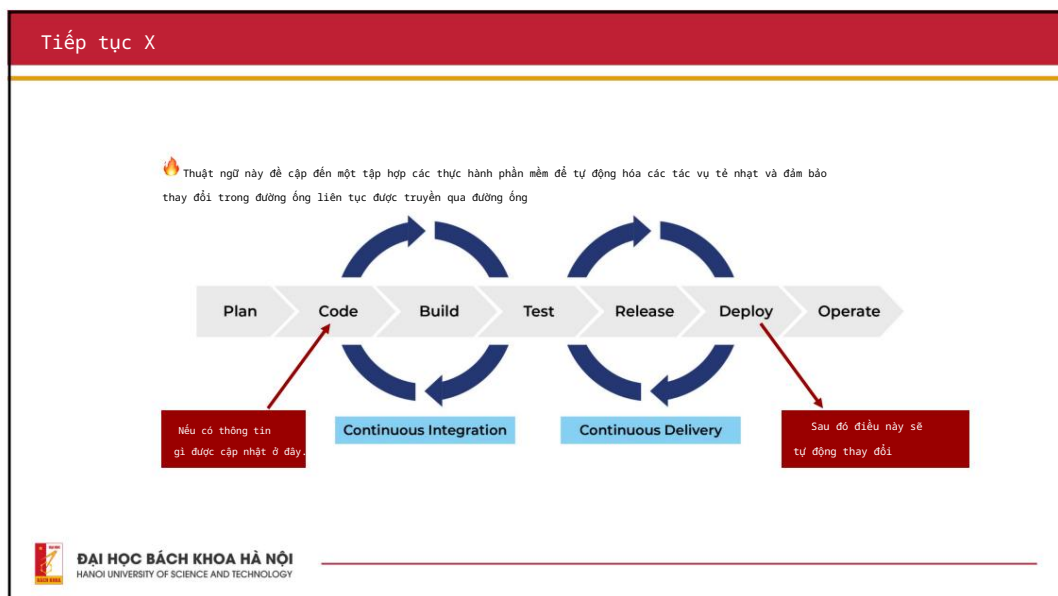


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

16



1



2

CI

Tiếp tục tích hợp

Nhiệm vụ cốt lõi:
💡 Làm thế nào để tự động bảo mật mã đó không bị hỏng trong quá trình phát triển?

🖥️ Khái niệm độc lập ứng dụng

đĩa CD

Tiếp tục triển khai

Nhiệm vụ cốt lõi:
💡 Làm thế nào để có được mã/ứng dụng của bạn người dùng tự động? + giám sát vòng đời

🖥️ Khái niệm phụ thuộc vào ứng dụng

CML

Tiếp tục học máy

Nhiệm vụ cốt lõi:
💡 Làm thế nào để tự động đào tạo lại máy mô hình học tập khi dữ liệu và mã thay đổi?

🖥️ Dành riêng cho các ứng dụng ML

OPERATIONS

→ No-Block Deployment

→ CI/CD Pipelines

→ Monitoring & Alerting

 **ĐẠI HỌC BÁCH KHOA HÀ NỘI**
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

3

Mức MLOps

Mô hình trường thành mô tả tổng thể các hoạt động DevOps để vận hành một môi trường MLOps thành công.

Nhằm xác định những lỗ hổng trong nỗ lực triển khai môi trường như vậy của một tổ chức hiện tại.

💡 Ước tính phạm vi công việc cho các cam kết mới.

💡 Thiết lập tiêu chí thành công thực tế.

💡 Xác định các mục tiêu bạn sẽ bàn giao khi kết thúc hợp đồng.

Level	Description	Highlights	Technology
0	No MLOps	- Difficult to manage full machine learning model lifecycle - The teams are disparate and releases are painful - Most systems exist as "black boxes," little feedback during/post deployment	- Manual builds and deployments - Manual testing of model and application - No centralized tracking of model performance - Training of model is manual
1	DevOps but no MLOps	- Releases are less painful than No MLOps, but rely on Data Team for every new model - Still limited feedback on how well a model performs in production - Difficult to trace/reproduce results	- Automated builds - Automated tests for application code
2	Automated Training	- Training environment is fully managed and scalable - Easy to reproduce model - Releases are manual, but low friction	- Automated model training - Centralized tracking of model training performance - Model management
3	Automated Model Deployment	- Releases are low friction and automatic - Full traceability from deployment back to original data - End-to-end managed: train > test > production	- Integrated A/B testing of model performance for deployment - Automated tests for all code - Centralized tracking of model training performance
4	Full MLOps Automated Operations	- Full system automated and easily monitored - Production systems are providing information on how to improve and, in some cases, automatically respond with new models - Approaching a zero-downtime system	- Automated model training and testing - Verboise, centralized metrics from deployed model

CI

đĩa CD

CML

 **ĐẠI HỌC BÁCH KHOA HÀ NỘI**
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

4

Tiếp tục tích hợp

Nhiệm vụ cốt lõi:

🔥 Làm thế nào để tự động bảo mật mã không bị hỏng trong quá trình phát triển? 🔥

3 bước để thực hiện điều này:

💡 Sử dụng kiểm soát phiên bản

Thường xuyên cam kết mã vào một kho lưu trữ được chia sẻ

💡 Viết (unit)test cho mã của bạn

Nên nắm bắt các lỗi không mong muốn trong mã của bạn

💡 Tự động xây dựng + thử nghiệm

Tự động chạy thử nghiệm để mã không thể được hợp nhất mà không hoạt động



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

5

Một nghiên cứu trường hợp nhỏ về tích hợp liên tục

The screenshot displays the GitHub repository for TorchMetrics. The commit history table is as follows:

File	Commit Message	Time Ago
MANIFEST.in	update CI	6 months ago
Makefile	rename tests/ (#1091)	5 months ago
README.md	Code cleaning after classification refactor 2/n (#1252)	7 days ago
pyproject.toml	CI: re-use checks (#1261)	2 months ago
requirements.txt	Set minimum pytorch version to 1.8 + cleanup (#1263)	2 months ago
setup.cfg	CI: re-use checks (#1261)	2 months ago
setup.py	CI: Enable testing with Python 3.10 (#1132)	5 months ago

The README.md section shows the TorchMetrics logo and the tagline: "Machine learning metrics for distributed, scalable PyTorch applications." It also includes links for "What is Torchmetrics", "Implementing a metric", "Built-in metrics", "Docs", "Community", and "License". At the bottom, there are badges for Python versions (3.7, 3.8, 3.9, 3.10), PyPI package status, downloads (23K), Conda status, and license (Apache 2.0).

On the right side, the "Contributors" section lists 158 contributors, and the "Languages" section shows Python at 99.0% and Other at 0.1%.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

6

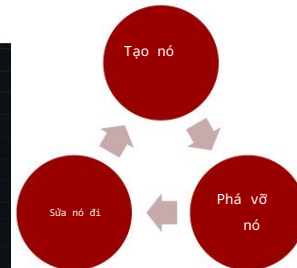
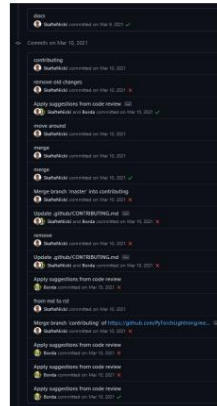
CI bước 1: kiểm soát phiên bản

Kiểm soát phiên bản người dùng:

- Thay đổi mã được theo dõi
- Các nhánh cho công việc song song

Cam kết thường xuyên:

- Phát hiện lỗi sớm hơn là muộn
- Đề nghị quay lại thời điểm mọi thứ đang hoạt động
- Việc hợp nhất có thể được thực hiện tự động



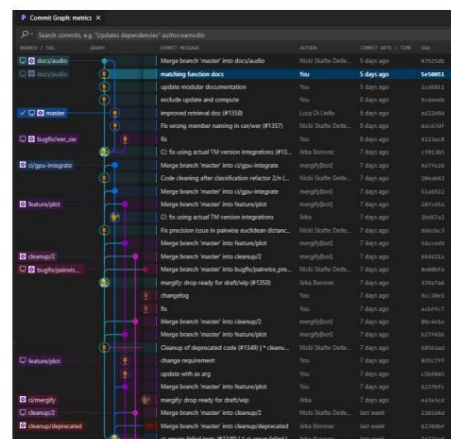
Bước 1 của CI: Sử dụng các nhánh

Quy trình làm việc song song

Các thay đổi tính năng thử nghiệm được giữ tránh xa bản chính/bản chính

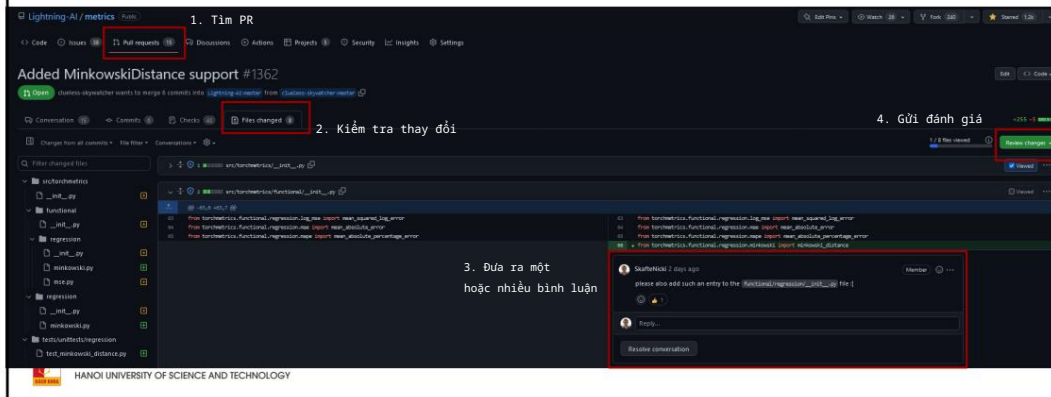
Đề xuất tiện ích mở rộng cho mã VS:

- [Gitlens hoặc GitGraph](#)
- [Github PR và các vấn đề](#)



Bước 1 của CI: Sử dụng yêu cầu kéo

⚡ Không thể đẩy bất kỳ cam kết nào lên master nếu không có trong yêu cầu kéo

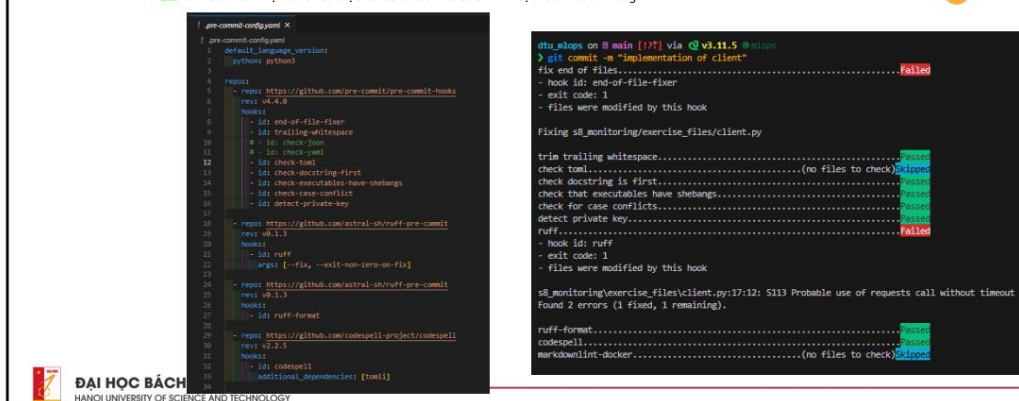


9

Bước 1 của CI: cam kết trước



✓ kiểm tra xem mọi thứ có đạt tiêu chuẩn trước khi tạo cam kết không



10

Bước 2 của CI: viết bài kiểm tra

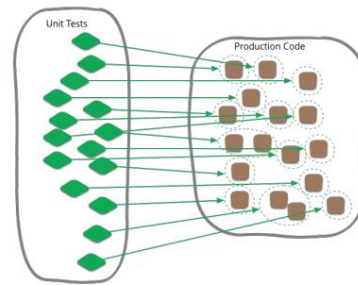
Kiểm thử là nền tảng của tích hợp liên tục, trong đó kiểm thử đơn vị được cho là quan trọng nhất.

Một unittest duy nhất, kiểm tra một phần nhỏ mã của bạn

Bằng cách kiểm tra mã thành từng phần nhỏ, lỗi sẽ dễ tìm hơn

Các loại thử nghiệm khác đáng cân nhắc:

- 🔥 Kiểm tra tích hợp
- 🔥 Kiểm tra hồi quy
- 🔥 Kiểm tra hiệu suất
- 🔥 Kiểm tra bảo mật



Bước 2 của CI: viết bài kiểm tra

Theo quy ước của Python, mã nguồn của bạn phải là

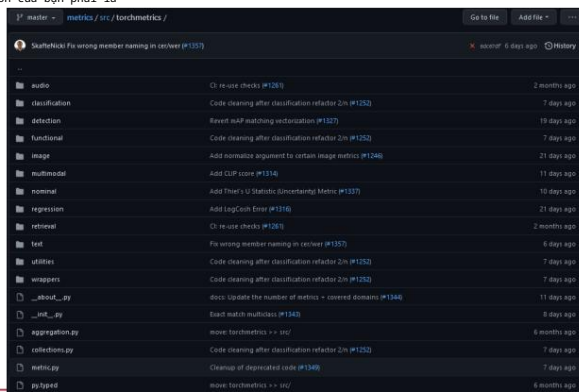
`src/<tên_dự_án>`

(bố cục src)

hoặc

`<tên_dự_án>`

(bố cục phẳng)



File/Directory	Commit Message	Time Ago
audio	CI re-use checks (#1287)	2 months ago
classification	Code cleaning after classification refactor 2in (#1252)	7 days ago
detection	Revert NMAP matching reclassification (#1207)	19 days ago
functional	Code cleaning after classification refactor 2in (#1252)	7 days ago
image	Add normalize argument to certain image metrics (#1246)	21 days ago
multimodal	Add CLIP score (#1314)	11 days ago
nominal	Add Tree's U Statistic (Uncertainty) Metric (#1375)	10 days ago
regression	Add LogCosh Error (#1316)	21 days ago
retinewr	CI re-use checks (#1287)	2 months ago
test	Fix wrong member naming in retinewr (#1357)	6 days ago
utils	Code cleaning after classification refactor 2in (#1252)	7 days ago
wrappers	Code cleaning after classification refactor 2in (#1252)	7 days ago
__about__.py	docs Update the number of metrics + covered domains (#1346)	11 days ago
__init__.py	Fast match multimodal (#1342)	8 days ago
aggregation.py	move torchmetrics > > src/	6 months ago
collections.py	Code cleaning after classification refactor 2in (#1252)	7 days ago
metric.py	Cleanup of deprecated code (#1348)	7 days ago
pytyped	move torchmetrics > > src/	6 months ago

Bước 2 của CI: viết bài kiểm tra

- Đối với các bài kiểm tra, cách thức thực hiện là đặt các bài kiểm tra vào một thư mục kiểm tra riêng biệt hoặc đặt các bài kiểm tra vào cùng thư mục với func, on/class/submodule mà chúng được kiểm tra.

```

• |— README.md
• |— src/
• |— init.py
  |— important_functions.py
• |— tests/
• |— init.py
  |— test_important_functions.py

```

```

• |— README.md
• |— src/
• |— init.py
  |— mô-đun con/
    |— init.py
    |— important_functions.py
  |— test_important_functions.py

```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

metrics / tests / unittests /			Go to file	Add file	...
6 authors Code cleaning after classification refactor 2/n (#1252) · up					
..					
audio	Set minimum pytorch version to 1.8 + cleanup (#1263)	2 months ago			
bases	Code cleaning after classification refactor 2/n (#1252)	7 days ago			
classification	Cleanup of deprecated code (#1248)	7 days ago			
detection	Revert nlp matching vectorization (#1327)	19 days ago			
helpers	Add CLIP score (#1314)	11 days ago			
image	Add normalize argument to certain image metrics (#1246)	21 days ago			
multimodal	Add CLIP score (#1314)	11 days ago			
nominal	Add Trier's U Statistic (Stochastically Metric) (#1327)	19 days ago			
paranoid	Fix precision issue in paranoid euclidean distance (#1352)	7 days ago			
regression	Add LogCosh Error (#1316)	21 days ago			
retrieval	Refactor Classification Un (#1254)	3 months ago			
test	Bugfix: Update message regarding connection issues to the HF hub (#1141)	5 months ago			
utilities	Set minimum pytorch version to 1.8 + cleanup (#1263)	2 months ago			
wrappers	Code cleaning after classification refactor 2/n (#1252)	7 days ago			
__init__.py	rename tests/ (#1081)	6 months ago			

13

Bước 2 của CI: viết bài kiểm tra

- 💡 Trong Python, chúng tôi khuyên bạn nên sử dụng framework pytest .
- 💡 Kiểm tra là các hàm đơn giản bắt đầu bằng test_ và sử dụng assert

```

import torch
from torch.nn.functional import mse_loss

def test_mse_loss_zeros():
    # (0 - 0)**2 = 0
    assert mse_loss(torch.zeros(1,), torch.zeros(1,)) == 0

def test_mse_loss_ones():
    # (1 - 0)**2 = 1
    assert mse_loss(torch.ones(1,), torch.zeros(1,)) == 0

```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

14

Bước 2 của CI: viết bài kiểm tra

Kiểm tra có thể đơn giản.

```
def test_warning_on_nan(tmpdir):
    preds = torch.randint(3, size=(20,))
    target = torch.randint(3, size=(20,))

    with pytest.warns(
        UserWarning,
        match='.* nan values found in confusion matrix have been replaced with zeros.',
    ):
        confusion_matrix(preds, target, num_classes=5, normalize='true')
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

15

Bước 2 của CI: viết bài kiểm tra

Kiểm tra có thể đơn giản.

```
def test_warning_on_nan(tmpdir):
    preds = torch.randint(3, size=(20,))
    target = torch.randint(3, size=(20,))

    with pytest.warns(
        UserWarning,
        match='.* nan values found in confusion matrix have been replaced with zeros.',
    ):
        confusion_matrix(preds, target, num_classes=5, normalize='true')
```

Hoặc phức tạp

```
@pytest.mark.parametrize("normalize", [True, 'pred', 'all', None])
@pytest.mark.parametrize(
    "preds, target, sk_metric, num_classes, multilabel",
    [
        (InputBinary_prob.preds, InputBinary_prob.target, sk_cm_binary_prob, 2, False),
        (InputBinary_logits.preds, InputBinary_logits.target, sk_cm_binary_prob, 2, False),
        (InputBinary_prob.preds, InputBinary_target, sk_cm_binary, 2, False),
        (Input_mlb_prob.preds, Input_mlb_prob.target, sk_cm_multilabel_prob, NUM_CLASSES, True),
        (Input_mlb_logits.preds, Input_mlb_logits.target, sk_cm_multilabel_prob, NUM_CLASSES, True),
        (Input_mlb.preds, Input_mlb.target, sk_cm_multilabel, NUM_CLASSES, True),
        (Input_mls_prob.preds, Input_mls_prob.target, sk_cm_multiclass_prob, NUM_CLASSES, False),
        (Input_mls_logits.preds, Input_mls_logits.target, sk_cm_multiclass_prob, NUM_CLASSES, False),
        (Input_mls.preds, Input_mls.target, sk_cm_multiclass, NUM_CLASSES, False),
        (Input_mmc_prob.preds, Input_mmc_prob.target, sk_cm_multidie_multiclass_prob, NUM_CLASSES, False),
        (Input_mmc.preds, Input_mmc.target, sk_cm_multidie_multiclass, NUM_CLASSES, False)]
)

class TestConfusionMatrix(MetricTester):
    @pytest.mark.parametrize("dop", [True, False])
    @pytest.mark.parametrize("dist_sync_on_step", [True, False])
    def test_confusion_matrix(
        self, normalize, preds, target, sk_metric, num_classes, multilabel, dop, dist_sync_on_step
    ):
        self.run_class_metric_test(
            dop=dop,
            preds=preds,
            target=target,
            metric_class=ConfusionMatrix,
            sk_metric=partial(sk_metric, normalize=normalize),
            dist_sync_on_step=dist_sync_on_step,
            metric_args={
                "num_classes": num_classes,
                "threshold": THRESHOLD,
                "normalize": normalize,
                "multilabel": multilabel
            }
        )
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

16

Bước 2 của CI: thực hiện cục bộ

```
(lightning) C:\Users\vsde\Documents\metrics\pytest tests\unittests\regression\test_mean_error.py
===== test session starts =====
platform win32 -- Python 3.8.13, pytest-6.2.5, py-1.11.0, pluggy-1.0.0
rootdir: C:\Users\vsde\Documents\metrics, configfile: setup.cfg
plugins: cov-4.0.0, doctestplus-0.12.1, timeout-2.1.0
collected 116 items

tests\unittests\regression\test_mean_error.py .....XXXXXXXXXXXXXXXXXXXXX

----- warnings summary -----
..\Anaconda3\envs\lightning\lib\site-packages\pytest\config\_init_.py:1183
C:\Users\vsde\Anaconda3\envs\lightning\lib\site-packages\pytest\config\_init_.py:1183: PytestDeprecationWarning: The --strict option is deprecated, use --strict-markers instead.
  self.issue_config_time_warning(
-- Docs: https://docs.pytest.org/en/stable/warnings.html
===== 98 passed, 28 skipped, 8 xfailed, 1 warning in 16.44s =====
```

- ✓ Test passed
- F Test failed
- S Test skipped (pytest.skipif, pytest.skip)
- X Test was expected to fail (pytest.xfail)

Bạn có nhớ thực hiện việc này trước mỗi lần commit không?

Hãy tự động hóa việc thực hiện nó thay thế



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

17

Bước 3 của CI: Tự động hóa mọi thứ

Những gì có thể tự động hóa: MỌI THỨ 

-  Kiểm thử đơn vị
-  Kiểm thử tích hợp
-  Tạo tài liệu
-  Linters (định dạng kiểu)
-  Kiểm tra an ninh
-  Phạm vi mã
-  Kiểm tra tùy chỉnh.

Chỉ có trí tưởng tượng của bạn là giới hạn.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

18

CI bước 3: Github actions

Tích hợp liên tục trong Github.

Miễn phí 2000 phút tự động hóa/tháng (kho lưu trữ công khai)

SkaftNicki / 02457_mlopsPrivate

<> Code🕒 Issues🔗 Pull requests🔧 Actions📁 Projects🛡 Security📊 Insights⚙ Settings

Nhiều quy trình làm việc sẵn sàng

Demo

By Github Actions

Test your demo project

Set up this workflow

Python package

By Github Actions

Create and test a Python package on multiple Python versions

Set up this workflow

Elabor

By Github Actions

Build and test an Elab project with M4

Set up this workflow

NET

By Github Actions

Build and test a .NET or ASP.NET Core project

Set up this workflow

Publish Docker Container

By Github Actions

Build, test and push Docker image to Github Packages

Set up this workflow

CI/CD

By Github Actions

Build and test a CI/CD project with Jenkins

Set up this workflow

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Bước 3 của CI: các tệp quy trình làm việc được kích hoạt

Trình làm việc nên được

Định nghĩa OS + python

Mã bản sao

Thiết lập Python

Cài đặt các phụ thuộc

Kiểm tra định dạng

Chạy thử nghiệm

Tệp quy trình làm việc là một tập hợp các hướng dẫn cần phải được thực hiện trên một máy ảo được lưu trữ bởi Github

Bạn có thể có một hoặc nhiều tệp quy trình công việc (chạy song song)

1name: Python package

2

3on:

4push:

5branches: [main]

6pull_request:

7branches: [main]

8

9jobs:

10build:

11

12runs-on: ubuntu-latest

13strategy:

14matrix:

15python-version: ["3.7", "3.8", "3.9", "3.10"]

16

17steps:

18- uses: actions/checkout@v3

19- name: Set up Python \${{ matrix.python-version }}

20uses: actions/setup-python@v4

21with:

22python-version: \${{ matrix.python-version }}

23- name: Install dependencies

24run: |

25python -m pip install --upgrade pip

26pip install flake8 pytest

27pip install -r requirements.txt

28python setup.py install

29- name: Lint with flake8

30run: |

31flake8 src/

32- name: Test with pytest

33run: |

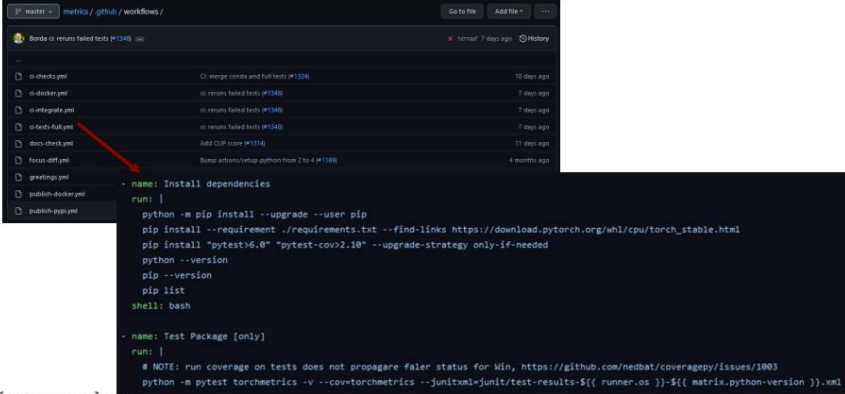
34pytest tests/

35

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

CI bước 3: tập tin quy trình làm việc



```
- name: Install dependencies
  run: |
    python -m pip install --upgrade --user pip
    pip install --requirement ./requirements.txt --find-links https://download.pytorch.org/whl/cpu/torch_stable.html
    pip install "pytest>6.0" "pytest-cov>2.10" --upgrade-strategy only-if-needed
    python --version
    pip list
    shell: bash

- name: Test Package [only]
  run: |
    # NOTE: run coverage on tests does not propagate folder status for Win, https://github.com/nedbat/coveragepy/issues/1003
    python -m pytest torchmetrics -v --cov=torchmetrics --junitxml-junit/test-results-${{ runner.os }}-${{ matrix.python-version }}.xml
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

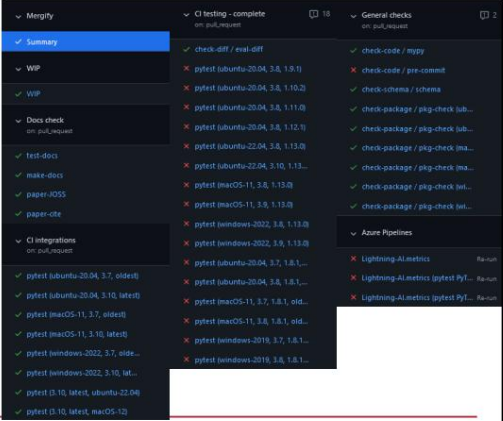
Bước 3 của CI: Tập quy trình làm việc

 **Đã cộng 43 lần kiểm tra**

Kiểm tra sự kết hợp của

-  Thiết lập phân cứng
-  Hệ điều hành
-  Phiên bản Python
-  Phụ thuộc cốt lõi

Chạy thử nghiệm đơn vị, xây dựng tài liệu, phạm vi thử nghiệm, kiểm tra mã, cài đặt gói, v.v.





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

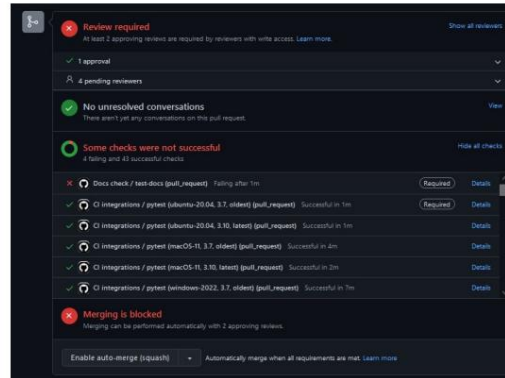
Bước 3 của CI: Kiểm tra mã trước khi hợp nhất

Quy tắc bảo vệ nhánh: Tất cả/

☛ một số bài kiểm tra phải vượt qua
☛ ít nhất x thành viên cốt cán cần

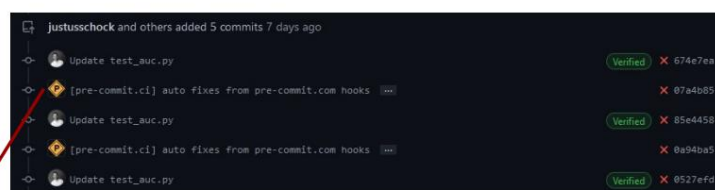
để chấp thuận
☛ bình luận phải được xử lý

Xem thêm [tại đây](#)



23

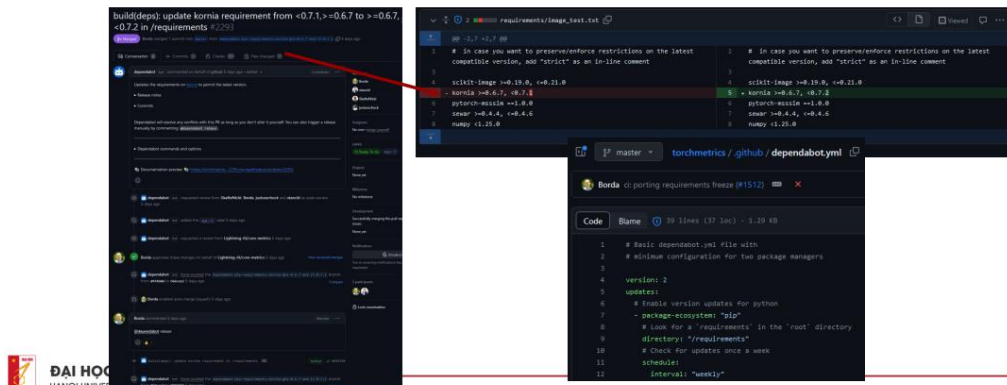
Bước 3 của CI: Tự động hóa các tác vụ lặp đi lặp lại bằng bot



24

Bước 3 của CI: Tự động hóa các tác vụ lặp đi lặp lại bằng bot

Dependabot có thể giúp tự động kiểm tra các bản phát hành mới của các phụ thuộc trong dự án của bạn



25

Tóm tắt về sự tích hợp liên tục

1. Sử dụng kiểm soát phiên bản
2. Viết (unit-)test cho mã của bạn
3. Tự động xây dựng + thử nghiệm



26



1



2

HỌC MÁY LIÊN TỤC

3

Tích hợp liên tục cổ điển

- Các phiên trước đã đề cập đến tích hợp liên tục cổ điển từ
Hoạt động phát triển.
- Các khái niệm được áp dụng cho quy trình học máy nhưng có thể áp dụng cho bất kỳ ứng dụng nào.
- Bây giờ, chúng tôi chuyển trọng tâm sang học máy liên tục (CML).



4

Học máy liên tục (CML)

- Tập trung vào việc tự động hóa các quy trình học máy.
 - Các nguyên tắc tích hợp liên tục là cần thiết để đảm bảo độ tin cậy của mô hình:
 - Tôi đã đào tạo trên dữ liệu chính xác chưa?
 - Mô hình của tôi có hội tụ không?
 - Nó có đạt đến ngưỡng không?
- CML cho phép kiểm tra liên tục, với các công cụ như cml từ [lập lại.ai](#).



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

5

Sử dụng CML trong quy trình làm việc

- cml hợp lý hóa tự động hóa, cung cấp báo cáo sau khi đào tạo.
 - Để kích hoạt đào tạo mô hình với mỗi lần đẩy git:
- chạy: `python train.py` • Quy trình làm việc sử dụng quy trình CI chuẩn:
- đẩy mã -> kích hoạt hành động GitHub -> thực hiện công việc.
 - Một báo cáo được tạo sau mỗi lần chạy.

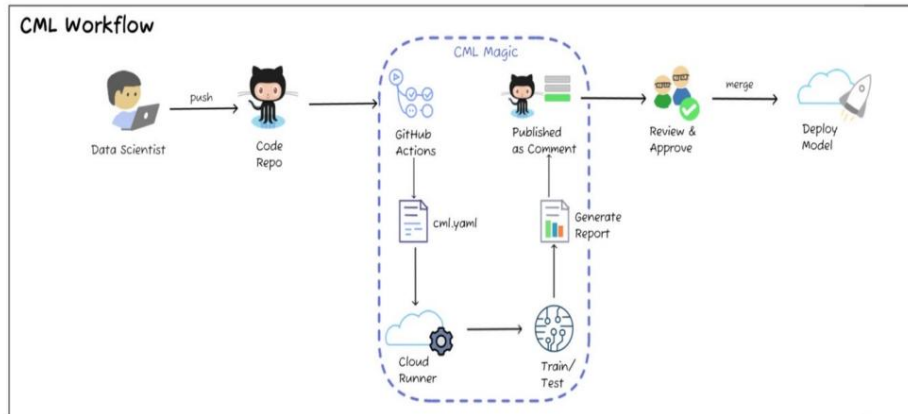


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

6

Tổng quan về quy trình làm việc CML

- Sơ đồ cho thấy toàn bộ quy trình sử dụng khung cml.



Nguồn hình ảnh: huongtroidatascience.com



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

7

Bài tập

1. Xem lại train.py •

Sửa đổi train.py thành train_cml.py để báo cáo hiệu suất sau khi đào tạo. • Thêm báo cáo ma trận độ chính xác và nhầm lẫn:

```
# giả sử chúng ta có một mô hình được đào tạo
import matplotlib.pyplot as plt from
sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay preds, target
= [], [] cho lô trong
train_dataloader: x, y = lô
probs = model(x)

preds.append(probs.argmax(dim=-1))
target.append(y.detach())

target = torch.cat(target, dim=0) preds =
torch.cat(preds, dim=0)

báo cáo = báo cáo phân loại (mục tiêu, dự đoán) với
open("classification_report.txt", 'w') là tệp đầu ra: outfile.write(báo cáo)
confmat = confusion_matrix(mục
tiêu, dự đoán) disp = ConfusionMatrixDisplay(ma trận nhầm
lẫn = confmat) plt.savefig('confusion_matrix.png')
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

8

2. Tạo quy trình làm việc CML

- Tạo tệp quy trình làm việc mới cml.yaml với nội dung sau:

```
tên: train-my-model vào lúc:
[đẩy]
công
việc: chạy:
  chạy trên: [ubuntu-latest]
  các
    bước: - sử dụng: actions/checkout@v2 -
    sử dụng: iterative/setup-cml@v1 - tên: Mô
    hình tàu hỏa
  chạy: |
    pip install -r requirements.txt # cài đặt các phụ thuộc
    python train.py # chạy training - name: Viết báo cáo env:

    REPO_TOKEN: ${ secrets.GITHUB_TOKEN } chạy: |

    phân loại mào_report.txt >> report.md
    cml-publish confusion_matrix.png --md >> report.md
    cml-gửi-bình luận báo cáo.md
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

9

3. Đẩy tệp quy trình công việc

- Đẩy cml.yaml lên GitHub và đảm bảo quy trình công việc hoàn tất.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

10

4. Gửi yêu cầu kéo • Xem

[video](#) này về [cách gửi](#) yêu cầu kéo. • Đảm bảo quy trình làm việc tạo báo cáo hiệu suất trong GitHub bình luận.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

11

5. (Tùy chọn) Tìm hiểu sâu hơn về cml và dvc

- Tìm hiểu cách cml tích hợp với dvc [tại đây](#).



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

12

CML và giới hạn phần cứng

- Trình chạy mặc định của GitHub Actions là máy chỉ sử dụng CPU.
- Học máy hiện đại thường yêu cầu GPU để đào tạo hợp lý lần.
- Sau khi tìm hiểu về điện toán đám mây, hãy thử nghiệm với [tự lưu trữ người chạy bộ](#).



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

13

Liên tục
Học hỏi

14

14

Hiệu suất của mô hình giảm sút trong quá trình sản xuất

- Sự thay đổi phân phối dữ liệu
 - Đột ngột
 - Tuần hoàn
 - Dần dần



15

Từ Giám sát đến Học tập liên tục

- Giám sát: phát hiện sự thay đổi phân phối dữ liệu • Học tập liên tục: liên tục điều chỉnh các mô hình để phù hợp với sự thay đổi phân phối dữ liệu



16

Học tập liên tục

- Thiết lập cơ sở hạ tầng sao cho các mô hình có thể liên tục học hỏi từ dữ liệu mới trong quá trình sản xuất • Đào tạo có trạng thái



17

Học tập liên tục: các trường hợp sử dụng

- Sự kiện hiếm hoi
 - Mua sắm Giáng sinh/Thứ sáu đen tối/Ngày Prime • Cảnh quan tổng thể • Khởi động lạnh
- liên tục (điều chỉnh trong phiên)
 - Người dùng mới
 - Thiết bị mới
 - Người dùng không đăng nhập
 - Người dùng hiếm khi đăng nhập



18

Học tập liên tục đặc biệt tốt cho

- Nhân tự nhiên: ví dụ người dùng nhấp chuột -> dự đoán tốt
- Vòng phản hồi ngắn
- Ví dụ:
 - RecSys
 - Xếp hạng
 - Dự đoán CTR của quảng cáo
 - eDiscovery



19

Tần suất đào tạo lại mô hình của bạn chỉ là một nút
vặn để xoay



20

Bạn nên cập nhật mô hình của mình thường xuyên như thế nào?

- Rất ít công ty thực sự cập nhật các mô hình với mỗi mẫu mới
 - Quên lãng thảm khốc • Có thể tốn kém không cần thiết*
- Cập nhật các mô hình với các lô nhỏ



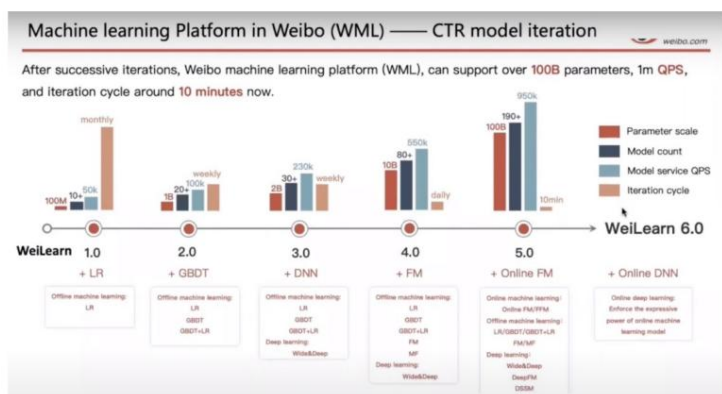
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

21

21

Chu kỳ lặp lại: phút

- Alibaba: Khuyến mãi Ngày Độc thân •
[Weibo](#) • [Tiktok](#)
 • [SheIN](#)



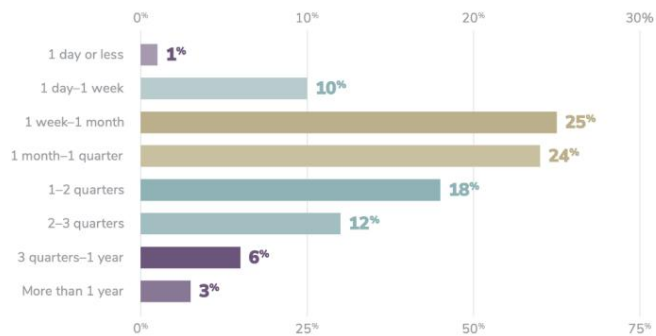
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

22

22

Chu kỳ lặp lại: Hoa Kỳ

Only 11% of organizations can put a model into production within a week, and 64% take a month or longer



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
Hình ảnh học máy doanh nghiệp của Algorithmia (2021)

23

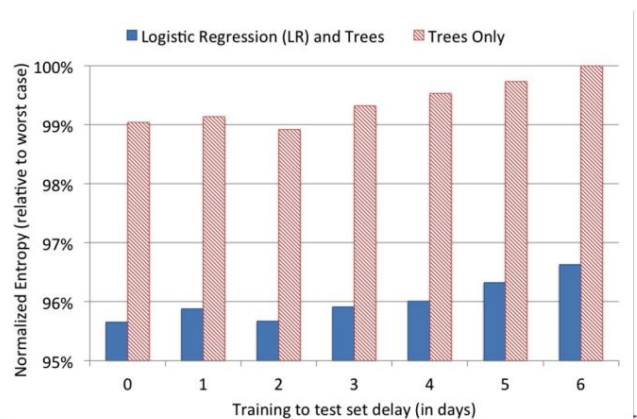
23

Định lượng giá trị của độ mới của dữ liệu

- Hiệu suất của mô hình thay đổi bao nhiêu nếu chuyển từ đào tạo lại hàng tháng sang hàng tuần sang hàng ngày theo giờ?

a. FB: Tỷ lệ CTR mất đi có thể giảm ~1%

chuyển từ đào tạo hàng tuần sang hàng ngày



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
Bài học thực tế từ việc dự đoán lượt nhấp chuột vào quảng cáo trên Facebook (2014)

24

24

Định lượng giá trị của độ mới của dữ liệu

1. Hiệu suất của mô hình thay đổi bao nhiêu nếu chuyển từ đào tạo lại hàng tháng sang hàng tuần sang hàng ngày theo giờ?
2. Khả năng ghi nhớ sẽ thay đổi như thế nào nếu bạn có thể điều chỉnh trong buổi học?



25

Định lượng giá trị của độ mới của dữ liệu

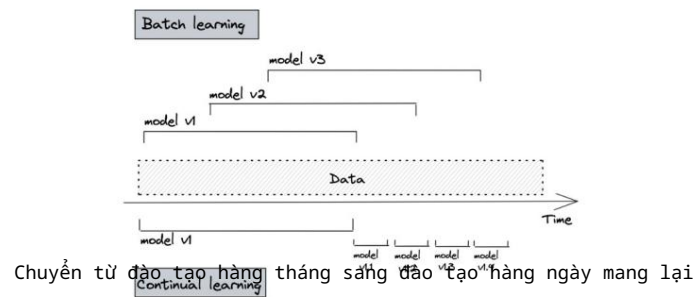
1. Hiệu suất của mô hình thay đổi bao nhiêu nếu chuyển từ đào tạo lại hàng tháng sang hàng tuần sang hàng ngày theo giờ?
2. Khả năng ghi nhớ sẽ thay đổi như thế nào nếu bạn có thể điều chỉnh trong buổi học?
3. Lập lại mô hình so với lập lại dữ liệu



26

Định lượng hóa đơn tiết kiệm đám mây

- Đào tạo mô hình tăng dần mỗi ngày trên dữ liệu mới •
- Hội tụ nhanh hơn cần ít tính toán hơn



GRUBHUB™

Tiết kiệm chi phí gấp 45 lần và tăng số liệu +20%



ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

[học trực tuyến để được giới thiệu tại Grubhub \(Alex Egg, 2021\)](#)

27

27

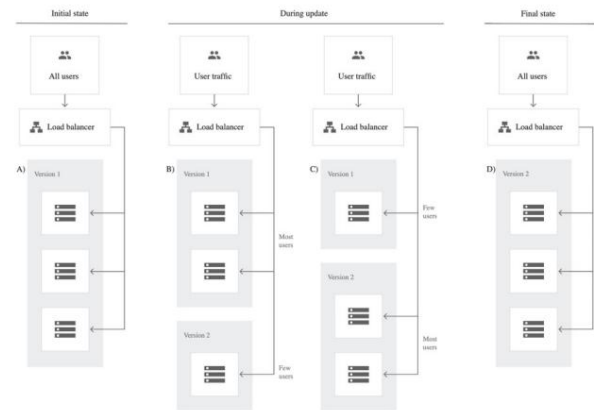
Kiểm tra trong
Sản xuất

30

30

Kiểm tra Canary

- Mô hình mới bên cạnh hệ thống hiện có
- Một số lưu lượng được định tuyến đến mô hình mới
- Tăng dần lưu lượng đến mô hình mới
- Ví dụ triển khai ở Việt Nam trước, sau đó là Châu Á, sau đó phần còn lại của thế giới



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

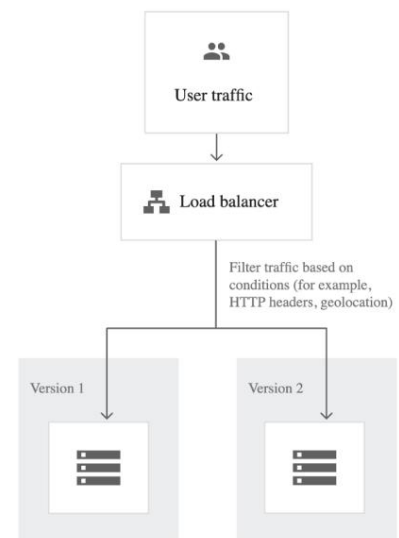
Chiến lược triển khai và thử nghiệm ứng dụng (Google)

31

31

Kiểm tra A/B

- Mô hình mới bên cạnh hệ thống hiện có
- Một phần trăm lưu lượng được định tuyến đến mô hình mới dựa trên các quy tắc định tuyến
- Kiểm soát đối tượng mục tiêu và giám sát bất kỳ sự khác biệt có ý nghĩa thống kê trong hành vi của người dùng
- Có thể có nhiều hơn 2 phiên bản



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Chiến lược triển khai và thử nghiệm ứng dụng (Google)

32

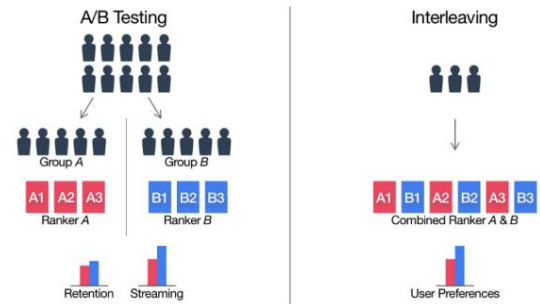
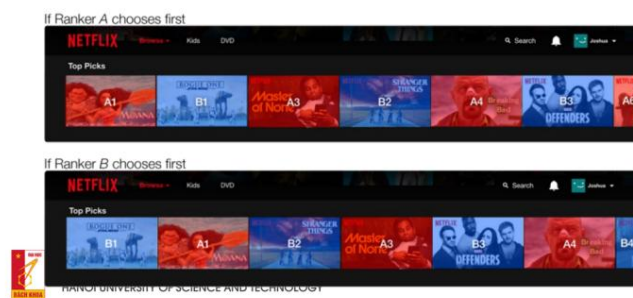
32

Thí nghiệm xen kẽ

- Đặc biệt hữu ích cho việc xếp hạng/recsys • Lấy các khuyến nghị từ cả hai mô hình A

& B

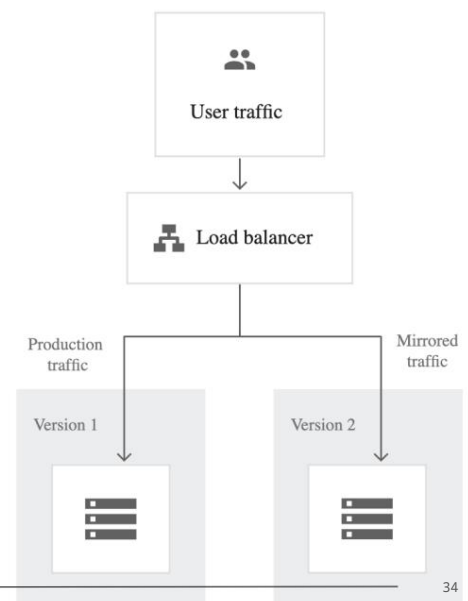
- Trộn chúng lại với nhau và hiển thị cho người dùng • Xem những đề xuất nào được nhấp vào



33

Kiểm tra bóng tối

- Mô hình mới song song với hệ thống hiện có • Các dự đoán của mô hình mới được ghi lại, nhưng không hiển thị cho người dùng
- Chuyển sang mô hình mới khi có kết quả thỏa đáng



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Chiến lược triển khai và thử nghiệm ứng dụng (Google)

34

Kiểm tra nội bộ trước

- Sử dụng các tính năng ngay cả khi chúng đang được phát triển
- Chia sẻ nội bộ trước khi chia sẻ ra bên ngoài

⚠️ Bạn và đồng nghiệp của bạn không phải là người dùng thông thường ⚠️



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

35

35

HUST

CẢM ƠN !



hust.edu.vn



fb.com/dhbkhn

36

36

Hành động Github

ONE LOVE. ONE FUTURE.

1

1

Tự động kiểm tra và hợp nhất mã an toàn

- Kiểm tra mã cục bộ có thể phức tạp do:
 - Cần phải chạy thường xuyên để phát hiện lỗi sớm
 - Kiểm tra mở rộng để có độ bao phủ mã cao, có thể mất thời gian
- Tự động hóa các bài kiểm tra cho mỗi lần đẩy kho lưu trữ có thể giúp ích
 - Việc hợp nhất các nhánh chỉ sau khi vượt qua các bài kiểm tra tự động đảm bảo mã an toàn hơn, giả sử các bài kiểm tra được bảo vệ tốt



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

2

2

Sử dụng GitHub Actions để tích hợp liên tục

- GitHub Actions cung cấp giải pháp tích hợp liên tục. • Mỗi kho lưu trữ nhận được 2.000 phút thử nghiệm miễn phí mỗi tháng, đủ cho các dự án cá nhân và khóa học.
- Thiết lập ban đầu của GitHub Actions có vẻ phức tạp. • Các tệp quy trình công việc được tạo cho một kho lưu trữ có thể được sử dụng lại cho các kho lưu trữ khác.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

3

3

Tổ chức tệp quy trình làm việc GitHub

- Bắt đầu bằng cách đặt tên cho quy trình công việc. • Chỉ định các sự kiện kích hoạt cho quy trình công việc: • Bao gồm các hành động (ví dụ: yêu cầu kéo, đẩy). • Xác định các nhánh áp dụng. • Liệt kê các công việc sẽ được thực hiện: • Theo mặc định, các công việc chạy song song nhưng có thể có các phụ thuộc. • Sử dụng run-on để chỉ định hệ điều hành cho quy trình công việc. • Xác định các bước, trong đó các lệnh thực tế sẽ được chạy được chỉ định.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

4

4

```

name: testing
on:
  pull_request:
    branches:
      - main
jobs:
  test-code:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repo
        uses: actions/checkout@v2
      - name: Install dependencies
        run: make install-test
      - name: Execute tests
        run: pytest tests/tagifai

```

- ◀ Event (single or multiple)

schedule: * * * * *
manual
on: push

on: pull_request
on: issues
...
- ◀ Jobs (In parallel or dependent)
- ◀ Runner
- ◀ Steps
- ◀ Actions (from Marketplace, custom, etc.)
- ◀ Commands

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

5

5

Bài tập

1. Tạo một thư mục .github trong thư mục gốc của kho lưu trữ của bạn và thêm một thư mục con thư mục có tên là quy trình công việc.
2. Xem lại [hướng dẫn GitHub Actions dành cho Python \(tập trung vào tự động kiểm tra mã Python\)](#)
3. Đặt tệp quy trình công việc được cung cấp có tên tests.yaml vào Thư mục .github/workflows/ .
4. Định nghĩa requirements.txt và requirements_tests.txt 5. Đẩy thay đổi vào kho lưu trữ và chạy thử nghiệm 6. Mở rộng thử nghiệm CI cho nhiều phiên bản hệ điều hành và Python 7. Triển khai bộ nhớ đệm trong quy trình làm việc 8. Thêm phạm vi mã vào quy trình làm việc 9. Thiết lập quy tắc bảo vệ nhánh 10. Kiểm tra dữ liệu tích hợp liên tục

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

6

6

Tập quy trình làm việc

- Đặt tập quy trình công việc được cung cấp có tên tests.yaml vào Thư mục .github/workflows/. Tập quy trình công việc này bao gồm ba bước:

- Khởi tạo môi trường Python (Python 3.8 trong trường hợp này).
- Cài đặt tất cả các phần phụ thuộc cần thiết để chạy thử nghiệm.
- Gọi pytest để thực hiện các thử nghiệm.

Xem lại tập để hiểu cấu trúc và cú pháp tổng thể của nó.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

7

7

Ví dụ về tests.yaml

```
tên: "Chạy thử nghiệm"

TRÊN:

đẩy:nhánh:      [ chủ, chính ]
pull_request:
  nhánh: [ master, main ]

việc làm:
  xây dựng:

    chạy trên: ubuntu-latest

    các bước:
      - tên: Thanh toán
        sử dụng: actions/checkout@v2 - tên:
        Thiết lập Python 3.8
        sử dụng: hành động/setup-python@v2
      với:
        python-version: 3.8 - name:
        Cài đặt các phụ thuộc chạy: |
          python -m pip cài đặt --nâng cấp pip
          pip cài đặt -r requirements.txt
          -r requirements_tests.txt - name:
        Kiểm tra bằng pytest chạy: |
          pytest -v
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

8

8

4. Định nghĩa requirements.txt và requirements_tests.txt

- requirements.txt: • Bao gồm tất cả các gói cần thiết để chạy mã. •
- requirements_tests.txt: • Bao gồm các gói bổ sung cần thiết để chạy thử nghiệm. • Có thể để trống nếu không cần thêm gói thử nghiệm nào. • Được sử dụng khi không cần các thư viện thử nghiệm cụ thể cho mã chính chức năng.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

9

9

5. Đẩy thay đổi vào kho lưu trữ và chạy thử nghiệm

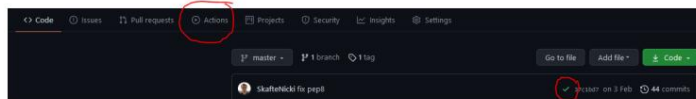
Đẩy các thay đổi vào kho lưu trữ.

Các bài kiểm tra sẽ tự động bắt đầu sau khi đẩy.

Sẽ có dấu kiểm màu xanh lá cây bên cạnh mã băm xác nhận sau khi các bài kiểm tra vượt qua.

Kiểm tra tab Hành động : Xem

lịch sử các hành động đã chạy.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

10

10

6. Mở rộng các bài kiểm tra CI cho nhiều phiên bản hệ điều hành và Python

1. Tập tests.yaml được cung cấp chạy trên một hệ điều hành:

- Có thể là ubuntu-latest.

2. Để thực hiện các thử nghiệm trên hai hệ điều hành chính khác: • Sử dụng thuộc tính strategy để xác định ma trận cho nhiều hệ điều hành.

```

công
việc: xây dựng:
  chạy trên: ${{ matrix.os }} chiến lược: matrix:
  os: ["ubuntu-
    latest",
      "windows-latest", "macos-latest"]

```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

11

11

6. Mở rộng các bài kiểm tra CI cho nhiều phiên bản hệ điều hành và Python (2)

3. Để chạy thử nghiệm trên các phiên bản Python khác nhau: • Thêm ma trận

```

python-version. jobs: build: runs-on: $
  ${{ matrix.os }}
  strategy: matrix: os: ["ubuntu-latest", "windows-
    latest", "macos-
    last"] python-
    version: [3.10, 3.11, 3.12] steps: - uses: actions/checkout@v4 - name: Thiết lập Python
  uses: actions/
    setup-python@v5 with: python-version: ${{ matrix.python-version }}

```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

12

12

7. Triển khai bộ nhớ đệm trong quy trình làm việc

- Các hành động của GitHub sẽ hủy mọi gói đã tải xuống khi quy trình công việc đã được thực thi
- 1. Để triển khai bộ nhớ đệm trong tệp quy trình làm việc của bạn, hãy làm theo các bước dưới đây: • Sử dụng tùy chọn bộ nhớ đệm trong hành động setup-python để lưu trữ bộ nhớ đệm các phụ thuộc. các bước:
 - sử dụng: `actions/checkout@v4` - sử dụng: `actions/setup-python@v5` với:

```
python-version: '3.10' cache:  
'pip' # Lưu trữ đệm các phụ thuộc pip
```
 - chạy: `pip install -r requirements.txt`



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

13

13

7. Triển khai bộ nhớ đệm trong quy trình làm việc (2)

2. Sau khi triển khai hệ thống lưu trữ đệm:
 - Vào Actions -> Caches trong kho lưu trữ của bạn để xác nhận bộ nhớ đệm đã được đã thêm.
 - Nó sẽ hiển thị các phụ thuộc được lưu trong bộ nhớ đệm như hình ảnh bên dưới:
3. Đo thời gian chạy quy trình công việc trước và sau khi thêm bộ đệm: • So sánh thời gian để xem bộ đệm có cải thiện thời gian chạy quy trình công việc hay không.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

14

14

8. Thêm phạm vi mã vào quy trình làm việc

Để thêm phạm vi mã vào quy trình làm việc của bạn, hãy làm theo các bước được nêu trong bài đăng được [liên kết](#):

1. Cài đặt coverage và codecov trong môi trường của bạn bằng cách thêm chúng vào requirements.txt hoặc cài đặt trực tiếp vào quy trình làm việc GitHub Actions của bạn.

```
pip cài đặt phạm vi bảo hiểm codecov
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

15

15

8. Thêm phạm vi mã vào quy trình làm việc

2. Cập nhật tệp quy trình làm việc của bạn để bao gồm các bước chạy phạm vi bảo hiểm và tải lên kết quả:

```
các
bước: - sử dụng: actions/checkout@v4 -
sử dụng: actions/setup-python@v5
với:
python-version: '3.10' - chạy:
pip install -r requirements.txt - chạy: coverage run
-m pytest # Chạy thử nghiệm với coverage - chạy: coverage xml # Tạo báo cáo
coverage theo định dạng XML - name: Tải coverage lên Codecov

sử dụng: codecov/codecov-action@v3
với:
file: ./coverage.xml # Tải lên báo cáo phạm vi phủ sóng
```

3. Đảm bảo kho lưu trữ của bạn được thiết lập với [Codecov](#) bằng cách đăng nhập và thêm dự án của bạn.
4. Sau khi triển khai, quy trình làm việc của bạn sẽ tải lên báo cáo về phạm vi mã như một hiện vật sau khi chạy thử nghiệm.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

16

16

9. Thiết lập Quy tắc bảo vệ nhánh

1. Tạo Bộ quy tắc nhánh:
2. Thiết lập Quy tắc Bảo vệ Nhánh
3. Triển khai Quy tắc Bảo vệ Nhánh với Admin Bypass



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

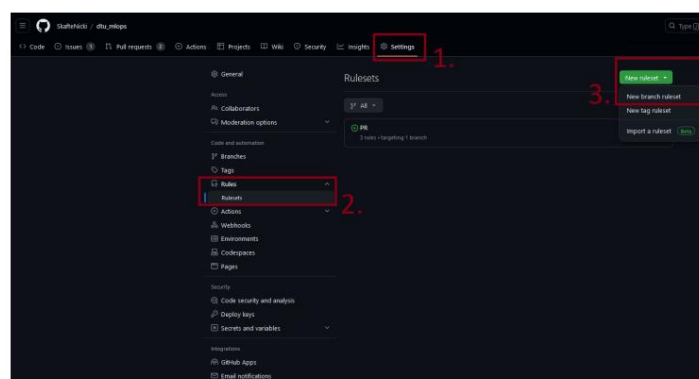
17

17

Tạo một bộ quy tắc nhánh:

Vào Cài đặt -> Quy tắc -> Bộ quy tắc trong kho lưu trữ của bạn.

Tạo một bộ quy tắc nhánh mới để xác định các điều kiện phải đáp ứng trước khi mã được hợp nhất vào nhánh chính.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

18

18

Thiết lập quy tắc bảo vệ nhánh

1. Đặt tên cho bộ quy tắc và đặt nhánh mục tiêu thành nhánh Mặc định (ví dụ: main hoặc master).
2. Các quy tắc chính cần xem xét:
 - Yêu cầu kéo trước khi hợp nhất:
 - Đảm bảo các thay đổi được hợp nhất thông qua các yêu cầu kéo.
 - Cho phép xem xét và thử nghiệm mã trước khi hợp nhất vào nhánh chính.
 - Tùy chọn, đặt yêu cầu cho số lượng người đánh giá hoặc phê duyệt cụ thể.
 - Yêu cầu kiểm tra trạng thái để vượt qua:
 - Đảm bảo tất cả các quy trình công việc đều vượt qua trước khi hợp nhất.
 - Chọn quy trình công việc nào là bắt buộc (ví dụ: thử nghiệm, phạm vi mã).



The screenshot shows the 'Require a pull request before merging' settings in a GitHub repository. The 'Required approvals' section is expanded, showing a requirement of 2 approvals. Below this, there are several checkboxes for additional rules, including 'Dismiss stale pull request approvals when new commits are pushed', 'Require review from Code Owners', 'Require approval of the most recent reviewable push', and 'Require conversation resolution before merging'. The 'Require status checks to pass' section is also expanded, showing a list of required status checks: 'Check Markdown formatting', 'Check Markdown links', and 'Check code formatting', each with a 'GitHub Actions' icon and a trash icon.



Triển khai Quy tắc bảo vệ nhánh với Admin Bypass

1. Các quy tắc bắt buộc để triển khai:

- Cần có ít nhất một người chấp thuận bất kỳ yêu cầu kéo nào.
- Tất cả các quy trình công việc phải được thông qua trước khi hợp nhất.
- Tất cả các cuộc hội thoại phải được giải quyết trước khi hợp nhất.

2. Bỏ qua quyền quản trị:

- Nếu các quy tắc có vẻ quá hạn chế, hãy cho phép quản trị viên kho lưu trữ bỏ qua các quy tắc.
- Thêm quản trị viên kho lưu trữ vào danh sách bỏ qua trong bộ quy tắc.

Điều này mang lại sự linh hoạt trong khi vẫn đảm bảo các bước kiểm tra và phê duyệt cần thiết được thực hiện trước khi mã được hợp nhất vào nhánh chính.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

21

21

3. Triển khai Quy tắc bảo vệ nhánh với Admin Bypass (2)

3. Triển khai thành công:

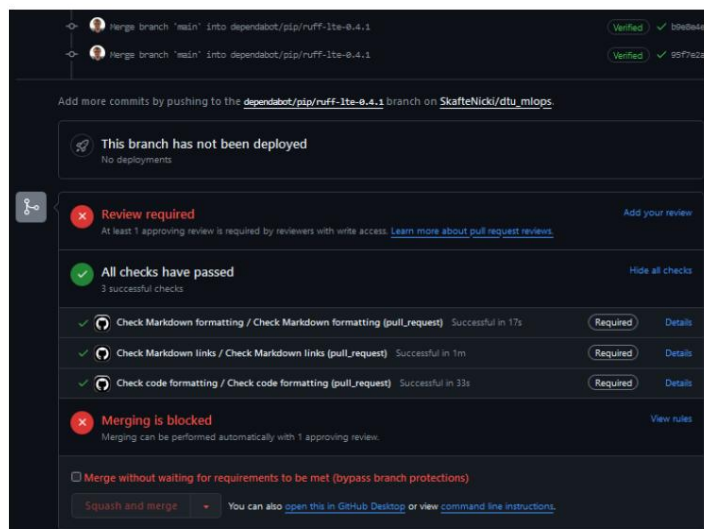
- Nếu các quy tắc được tạo đúng, bạn sẽ thấy thông báo tương tự như hình ảnh bên dưới khi cố gắng hợp nhất yêu cầu kéo.
- Cả ba lần kiểm tra phải vượt qua trước khi hợp nhất:
 - Ít nhất một lần được người đánh giá chấp thuận.
 - Tất cả quy trình công việc phải thành công.
 - Tất cả các cuộc hội thoại phải được giải quyết.
- Một quy tắc bỏ qua cho quản trị viên kho lưu trữ cũng phải được hiển thị, cho phép họ hợp nhất bất kể việc kiểm tra.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

22

22



23

23

10. Kiểm tra dữ liệu tích hợp liên tục

- Sự cố gặp phải: không thể chạy thử nghiệm do dữ liệu không được lưu trữ trên GitHub.
- Giả định: Mô-đun M8 - DVC đã hoàn thành.
- Vấn đề cốt lõi: Dữ liệu cần có sẵn để thử nghiệm.
- Giải pháp được đề xuất: Tải xuống dữ liệu trong quá trình tích hợp liên tục (CI) để cho phép thử nghiệm.

24

24

10.1 Vấn đề xác thực đường ống CI

- Đã xác định được vấn đề: Đường ống CI cần xác thực với bộ lưu trữ giải pháp.
- Giải pháp: Sử dụng tệp xác thực được tạo trong lần đẩy DVC đầu tiên.
- Vị trí tệp tin:

\$CACHE_HOME/pydrive2fs/{gdrive_client_id}/default.json

- macOS: ~/Thư viện/Bộ nhớ đệm
- Linux: ~/.cache (có thể thay đổi tùy theo bản phân phối)
- Windows: {user}/AppData/Local

- Nội dung mong đợi của tệp tin (ví dụ):

```
{  
  "access_token": ...,  
  "client_id": ...,  
  "client_secret": ...,  
  "refresh_token": ...,  
  ...  
}
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

25

25

10.2 Lưu trữ an toàn dữ liệu xác thực

- Quan trọng: Nội dung tệp xác thực phải được coi như mật khẩu và được giữ bí mật.
- Thách thức: Làm thế nào để sử dụng thông tin nhạy cảm trong kho lưu trữ công cộng.
- Giải pháp: Sử dụng bí mật của GitHub để lưu trữ dữ liệu nhạy cảm một cách an toàn.
- Các bước hành động:
 - Điều hướng đến tùy chọn bí mật GitHub.
 - Tạo một bí mật có tên là GDRIVE_CREDENTIALS_DATA.
 - Lưu trữ nội dung của tệp xác thực đã tìm thấy trước đó.
- Minh họa: Hình ảnh ví dụ được cung cấp hiển thị bí mật GitHub giao diện.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

26

26

10.3 Cập nhật và xác thực tệp quy trình làm việc

• Bước tiếp theo: Cập nhật tệp quy trình làm việc bằng mã sau: - `uses: iterative/setup-dvc@v1` - `name: Get data run: dvc pull env:`

```
GDRIVE_CREDENTIALS_DATA: ${ secrets:
  GDRIVE_CREDENTIALS_DATA }
```

• Mục đích: Mã này chạy lệnh `dvc pull` bằng tệp xác thực bí mật. • Tài nguyên bổ sung: Để được hướng dẫn, hãy truy cập kho [lưu trữ nhỏ](#) được cung cấp triển khai [quy trình làm việc](#) tương tự. • Các bước cuối cùng: • Thêm

thay đổi vào

kho lưu trữ. • Cam kết cập nhật.

• Đẩy các thay đổi. •

Xác nhận chức năng:

Đảm bảo rằng các bài kiểm tra đơn vị dựa trên dữ liệu đầu vào có thể chạy thành công.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

27

27

Thiết lập quy trình thực hành mã hóa

• Tạo tệp quy trình làm việc mới `codecheck.yaml`: • Thiết lập môi

trường Python. • Cài đặt `ruff`. • Chạy

`ruff check` và

`ruff format` trên kho lưu trữ.

• Thêm một bước để chạy `mypy` trên kho lưu trữ. • Đảm

bảo tất cả các bước đều

vượt qua: • Tập trung vào việc

vượt qua `ruff`. • Tùy chọn: Cố gắng vượt qua `mypy`, mặc dù điều này có thể khó khăn hơn.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

28

28



Kiểm tra kiến thức

- Bốn khái niệm chính trong GitHub Actions:
 - Quy trình làm việc: Tập YAML chỉ định hướng dẫn chạy trên các sự kiện nhất định, nằm trong thư mục `.github/workflows`.
 - Runner: Môi trường nơi các quy trình công việc thực thi, thường được lưu trữ bởi GitHub nhưng cũng có thể tự lưu trữ.
 - Công việc: Một tập hợp các bước được thực hiện trên cùng một trình chạy; quy trình công việc bao gồm ít nhất một công việc, thường là nhiều hơn.
 - Hành động: Đơn vị nhỏ nhất trong quy trình công việc; công việc bao gồm nhiều hành động được thực hiện tuần tự.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

29

29



Kiểm tra kiến thức

- Thuộc tính on chỉ định sự kiện nào mà quy trình công việc sẽ được kích hoạt. Giả sử bạn đã đặt thuộc tính on thành như sau:

TRÊN:

```

đẩy:
  nhánh: [chính] yêu cầu kéo:
  nhánh: [chính] lịch trình:
    - cron: "0 0 * * *"
  workflow_dispatch:
    {} • 4 sự kiện nào sẽ kích hoạt việc
    thực hiện hành động đó?
  
```

1. Đẩy trực tiếp vào nhánh chính sẽ kích hoạt nó
2. Bất kỳ yêu cầu kéo nào được mở sẽ hợp nhất vào nhánh chính sẽ kích hoạt nó
3. Vào cuối ngày, hành động sẽ kích hoạt, hãy xem `cron` để biết thêm thông tin
4. Có thể thực hiện kích hoạt bằng cách kích hoạt thủ công thông qua Giao diện người dùng GitHub, ví dụ, được hiển thị bên dưới



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

30

30

Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

New workflow

All workflows

Check code formatting

Check Markdown formatting

Check Markdown links

Deploy docs

pages-build-deployment

Run repo scraper

Management

Caches

Deployments

Run repo scraper

repo_scraper.yml

Help us improve GitHub Actions

13 workflow runs

This workflow has a workflow_dispatch event trigger.

Deployment of tools (#195)

Deployment of tools

Run workflow

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

31

Cam kết trước

1

1

Thực hành tốt nhất của Git Commit và Pre-Commit Hooks

- Thường xuyên cam kết quản lý những thay đổi nhỏ hơn, giúp việc hoàn nguyên dễ dàng hơn những thay đổi không mong muốn.
- Các nhiệm vụ tinh thần trước khi cam kết bao gồm lưu tệp, tạo kiểu, định dạng và đảm bảo các bài kiểm tra vượt qua.
- Các tác vụ này có thể xung đột với nguyên tắc cam kết thường xuyên. •

Các móc trước khi cam kết có thể tự động hóa các tác vụ này trong mỗi git làm.

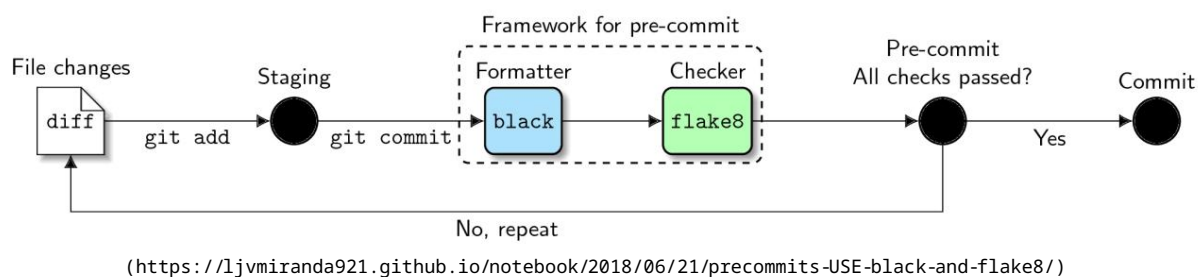


2

2

Cấu hình

- Pre-commit chỉ hoạt động bằng cách chèn bất kỳ quy trình công việc nào mà chúng ta muốn tự động hóa vào giữa bất cứ khi nào chúng ta thực hiện git commit và sau đó thực hiện git push.
- Hệ thống hoạt động bằng cách tìm kiếm tệp có tên `.pre-commit-config.yaml` mà chúng ta có thể cấu hình.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

3

3

Một tập tin mẫu

pre-commit sample-config | tệp ra `.pre-commit-config.yaml` -encoding utf8

Kết quả:

```

# Xem https://pre-commit.com để biết thêm thông tin
# Xem https://pre-commit.com/hooks.html để biết thêm các hook
kho lưu trữ:
- kho lưu trữ: https://github.com/pre-commit/pre-commit- hooks

sửa đổi: v3.2.0
móc: - id:
khoảng trắng theo sau
- id: end-of-file-fixer - id: check-yaml
- id: check-added-large-files
  
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

4

4

Tập cấu hình và thực thi trước khi cam kết

- Lệnh: `pre-commit sample-config | out-file .pre-commit-config.yaml -encoding utf8`
- Kết quả: Một tập YAML có cấu hình trước khi cam kết được tạo ra
- Cấu trúc tập YAML:
 - Liệt kê các kho lưu trữ cho các cam kết trước, ví dụ: <https://github.com/pre-commit/pre-commit-hooks>
 - Định nghĩa các hook trước khi commit bằng cách sử dụng id
 - Các hook ví dụ trong tập:
 - `trailing-whitespace`
 - `end-of-file-fixer`
 - `check-yaml`
 - `check-added-large-files`
 - id tương ứng với các mục trong `.pre-commit-hooks.yaml` được tìm thấy trong kho lưu trữ tại URL được cung cấp



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

5

5

Cài đặt cấu hình Pre-commit

- Sau khi xác định `.pre-commit-config.yaml`, hãy cài đặt nó bằng cách sử dụng:
`pre-commit install`
- Điều này đảm bảo các hook trước khi cam kết chạy tự động trên mỗi lần thực hiện cam kết git



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

6

6

? Bài tập

1. Cài đặt pip pre-commit trước khi cài đặt
2. Tiếp theo tạo tệp mẫu

```
pre-commit sample-config > .pre-commit-config.yaml
```
3. Tệp mẫu đã chứa 4 hook. Hãy đảm bảo bạn hiểu chức năng của từng loại và liệu bạn có cần chúng không.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

7

7

4. pre-commit hoạt động bằng cách móc vào lệnh git commit, chạy bất cứ khi nào lệnh đó được chạy. Để lệnh này hoạt động, chúng ta cần cài đặt các hook vào git commit. Chạy cài đặt trước để thực hiện việc này.
5. Thử commit file .pre-commit-config.yaml bạn vừa tạo. Bạn có thể sẽ không làm gì cả, vì pre-commit chỉ kiểm tra các file đang được commit. Thay vào đó, hãy thử chạy `pre-commit run --all-files` để kiểm tra mọi file trong kho lưu trữ của bạn.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

8

8

6. Hãy thử thêm ít nhất một kiểm tra khác từ kho lưu trữ cơ sở vào tệp `.pre-commit-config.yaml`.
7. Nếu bạn đã tìm hiểu về linter ruff. ruff đi kèm với pre-commit hook của riêng nó. Hãy thử thêm nó vào tệp `.pre-commit-config.yaml` của bạn và xem điều gì xảy ra khi bạn cố gắng commit tệp.
8. (Tùy chọn) Thêm nhiều hook hơn vào `.pre-commit-config.yaml` của bạn.



9. Đôi khi bạn vội vã, vì vậy hãy chắc chắn rằng bạn cũng có thể làm cam kết mà không chạy pre-commit ví dụ
`git commit -m <tin nhắn> --no-verify`
10. Cuối cùng, hãy tìm ra cách vô hiệu hóa precommit một lần nữa (nếu bạn cảm thấy mệt mỏi của nó).

